MEKELLE UNIVERSITY

MEKELLE INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

Advanced Database Systems

Module

All Chapters in One
Compiled Module

Compiled By: Mezgebe Mehari | To: 2nd Year IT Students

MU-MIT, Mekelle-Tigray
Semester-II, 2017 E.C

Table of Contents

CHAPTER - ONE: QUERY PROCESSING AND OPTIMIZATION *3*

1.1 Overview of Query Optimization 3

1.2 Translating SQL Queries into Relational Algebra 4

1.3 Basic Algorithms for Executing Query Operations 6

1.3.1 External Sorting 6

1.3.2 Implementing the SELECT Operation 7

1.3.3 Implementing the JOIN Operation 9

1.4 Using Heuristics in Query Optimization 11

1.4.1 Notation for Query Trees and Query Graphs 11

1.5 Using Selectivity and Cost Estimates in Query Optimization 12

1.5.1 Catalog Information Used in Cost Functions 14

1.6 Overview of Query Optimization in ORACLE 15

1.7 Semantic Query Optimization 17

CHAPTER – TWO: TRANSACTION PROCESSING *18*

2.1 Introduction to Transaction Processing 18

2.1.1 Single-User versus Multiuser Systems 18

2.1.2 Transactions, Read and Write Operations, and DBMS Buffers 19

2.2 Transaction Processing and System Concepts 21

2.2.1 Transaction States and Additional Operations 21

2.2.2 The System Log 22

2.3 Desirable Properties of Transactions 23

2.4 Schedules Based on Recoverability and Serializability 27

2.4.1 Schedules (Histories) of Transactions 27

2.4.2 Schedules Based on Recoverability and Serializability 28

2.5 Transaction Support in SQL 31

CHAPTER – THREE: CONCURRENCY CONTROL TECHNIQUES *34*

3.1 Locking Techniques for Concurrency Control 34

3.2 Concurrency Control Based on Timestamp Ordering 37

3.2.1 The Timestamp Ordering Algorithms 37

3.3 Multiversion Concurrency Control Techniques 39

3.3.1 Multiversion Technique Based on Timestamp Ordering 40

3.4 Validation (Optimistic) Concurrency Control Techniques 41
3.5 Granularity of Data Items 42
3.5.1 Granularity Level Considerations for Locking 43
3.6 Using Locks for Concurrency Control in Indexes 44
CHAPTER – FOUR: Database Recovery Techniques *45*
4.1 Recovery Concepts 45
4.2 Recovery Techniques Based on Deferred and Immediate Updates 48
4.2.1 Recovery Techniques Based on Deferred Update 48
4.3 Shadow Paging and The ARIES Recovery Algorithm 50
4.4 Recovery in Multidatabase Systems 53
CHAPTER – FIVE: DISTRIBUTED DATABASE SYSTEM *55*
5.1 Distributed Database Concepts, Data Fragmentation, Replication 55
5.2 Allocation Techniques for Distributed Database Design 55
5.3 Concurrency Control and Recovery in Distributed Databases 60
5.4 Transaction Management in Distributed Databases ..... **Error! Bookmark not defined.**
5.5 Types of Distributed Databases and Distributed DB in Oracle 63

CHAPTER - ONE: QUERY PROCESSING AND OPTIMIZATION

Query Processing and Optimization

1.1 Overview of Query Optimization

- In this chapter we will discuss the ***techniques*** used by a **DBMS** to process, *optimize*, and *execute* high-level queries.
- A query expressed in a high-level query language such as SQL must first be ***scanned, parsed, and validated***. The **scanner** identifies the language tokens—such as SQL keywords, attribute names, and relation names—in the text of the query, whereas the **parser** checks the query syntax to determine whether it is formulated according to the ***syntax rules*** (rules of grammar) of the query language.
- The query must also be **validated,** by checking that all attribute and relation names are valid and semantically meaningful names in the schema of the particular database being queried.
- An internal representation of the query is then created, usually as a tree ***data structure*** called a **query tree.** It is also possible to represent the query ***using a graph*** data structure called a **query graph.**
- The DBMS must then devise an **execution strategy** for retrieving the result of the query from the database files. A query typically has many possible execution strategies, and ***the process of choosing a suitable*** one for processing a query is known as **query optimization.**
- From **fundamentals of database** course of chapter 2.5, the ***runtime database processor*** has the task of running the query code, whether in **compiled** or **interpreted** mode, to produce the query result. If a ***runtime error results***, an error message is generated by the ***runtime database processor***.
- The term ***optimization*** is actually a **misnomer** because in some cases the chosen execution plan is not the **optimal** (best) strategy—it is just a ***reasonably efficient strategy*** for executing the query. Finding the optimal strategy is usually too **time-consuming except** for the simplest of queries and may require information on how the files are implemented and even on the contents of the files—information that may not be fully available in the DBMS catalog.
- Hence, ***planning of an execution strategy*** may be a more **accurate** description than ***query optimization***.
- If a DBMS provides only a navigational language, there is ***limited need or opportunity*** for extensive query optimization by the DBMS; instead, the programmer is given the capability to choose the "**optimal**" execution strategy. ***On the other hand***, a high-level query language—such as SQL for relational DBMSs (RDBMSs) or OQL for object DBMSs (ODBMSs)—is *more declarative in nature* because it specifies what the intended

results of the query are, rather than identifying the details of ***how*** the result should be obtained. Query optimization is thus necessary for queries that are specified in a ***high-level query*** language.

- A relational DBMS must *systematically* evaluate alternative query execution strategies and choose a *reasonably efficient or optimal* strategy. Each DBMS typically has a number of general database access algorithms that implement relational operations such as SELECT or JOIN or combinations of these operations. **Only execution strategies** that can be implemented by the DBMS access algorithms and that apply to the particular query and particular physical database design can be considered by the query optimization module.
- We will start in Section 1.2 with a general discussion *of how SQL queries are typically translated into relational algebra* queries and then optimized. We will then discuss algorithms for implementing relational operations in Section 1.3. Following this, we give an overview of query optimization strategies. There are *two main techniques for implementing query optimization*.
    - The first technique is based on **heuristic rules** for **ordering** the operations in a query execution strategy. A heuristic is a rule that works well in most cases but is ***not guaranteed to work well in every*** possible case. The rules typically reorder the operations in a query tree.
    - The second technique involves **systematically estimating** the cost of different execution strategies and choosing the execution plan with the ***lowest cost estimate***.
- The **two techniques** are usually combined in a query **optimizer**. We will discuss heuristic optimization in Section 1.2 in detail. We then provide a brief overview of the factors considered during query optimization in the ORACLE commercial RDBMS in Section 1.6. Section 1.7 introduces the topic of **semantic query optimization**, in which **known constraints** are used to devise efficient query execution strategies.

1.2 Translating SQL Queries into Relational Algebra

- In practice, ***SQL is the query language*** that is used in *most commercial RDBMSs*. An SQL query is **first** *translated* into *an equivalent extended relational algebra expression*—represented as a *query tree data structure*—that is then optimized.
- **Typically**, SQL queries are decomposed *into* ***query blocks*****,** which form the basic units that can be translated into the algebraic operators and optimized. A query block contains a single *SELECT-FROM-WHERE expression*, as well as *GROUP BY and HAVING* clauses if these are part of the block.
    - Hence, ***nested queries within a query*** are identified as separate query

blocks. **Because** SQL includes *aggregate operators*—such as MAX, MIN, SUM, AND COUNT—these operators must also be included in the extended algebra, as we discussed in Fundamentals of Database Systems.

- Consider the **following SQL query** on the **EMPLOYEE** relation:

```
SELECT LNAME, FNAME
FROM   EMPLOYEE
WHERE SALARY >  (SELECT   MAX (SALARY)
                 FROM     EMPLOYEE
                 WHERE    DNO=5);
```

- This query includes a **nested subquery** and hence would be decomposed ***into two blocks***.
  - The ***inner block*** is:

```
(SELECT MAX (SALARY)
FROM   EMPLOYEE
WHERE DNO=5)
```

  - And the ***outer block*** is:

```
SELECT LNAME, FNAME FROM
EMPLOYEE WHERE SALARY > C
```

  - Where **c *represents*** the result returned from the inner block.
  - The inner block could be translated into the extended relational algebra expression:

$$\text{MAX SALARY}\left(\sigma_{\text{DNO}=5}\left(\text{EMPLOYEE}\right)\right)$$

  - And the ***outer block*** into the expression:

$$\pi_{\text{LNAME, FNAME}}\left(\sigma_{\text{SALARY}>C}\left(\text{EMPLOYEE}\right)\right)$$

- The ***query optimizer*** would then choose an *execution plan for each block*. We should note that in the above example, the ***inner block needs*** to be evaluated **only once** to produce the *maximum salary*, which is then used—***as the constant c***—by the **outer** block.
  - We normally call such kind of nested query as an ***uncorrelated nested query***.
  - It is much harder to ***optimize the more complex correlated nested queries***,

where a *tuple variable from the outer block* appears in the **WHERE-clause** of the inner block.

1.3 Basic Algorithms for Executing Query Operations

- An **RDBMS** must include **algorithms** for implementing the different types of relational operations (*as well as other types of operations*) that can appear in a query ***execution strategy***.
  - These operations include the basic and extended relational algebra operations discussed in FDBS and, in many cases, combinations of these operations. For each such operation or combination of operations, one or more algorithms would typically be available to execute the operation(s).
- An **algorithm** may apply only to ***particular storage structures*** and ***access paths***; if so, it can only be used if the files involved in the operation include these access paths.
  - In this section we discuss typical algorithms used to implement ***SELECT***, ***JOIN***, and other ***relational*** operations. We begin by discussing *external sorting*, which is at the *heart of many relational* operations that utilize *sort-merge strategies.* Then we discuss algorithms for implementing the SELECT and JOIN operation.

1.3.1 External Sorting

- Sorting is one of the **primary algorithms** used in ***query processing***.
  - For example, whenever an SQL query specifies an ORDER BY-clause, the query result must be sorted.
- Sorting is ***also a key component in sort-merge algorithms*** used for ***JOIN*** and ***other*** operations (such as ***UNION*** and ***INTERSECTION***), and in duplicate elimination algorithms for the PROJECT operation (when an SQL query specifies the DISTINCT option in the SELECT clause).
- **External sorting** refers to sorting algorithms that are suitable for large files of records stored on disk that do not fit entirely in ***main memory***, such as ***most database*** files.
  - The typical external sorting algorithm uses a **sort-merge strategy**, which starts by sorting small **sub-files**—called **runs**—of the main file and then *merges the sorted* runs, creating larger sorted sub-files that are merged in turn.
- The ***sort-merge algorithm***, like other database algorithms, requires ***buffer space*** in main memory, where the ***actual sorting*** and ***merging*** of the runs is performed.
  - **It consists of two phases:**

1. The **sorting** phase and
2. The **merging** phase

- In the ***sorting phase*****,** runs (portions or pieces) of the file that can fit in the available buffer space are read into ***main memory***, sorted using an ***internal sorting algorithm***, and written back to disk as temporary sorted subfiles (or runs). The size of a run and ***number* of *initial runs*** is dictated by the ***number* of *file blocks* (*b*)** and the ***available buffer space*****.**

  - For example, if = 5 blocks and the size of the file *b* = 1024 blocks, then =, or 205 initial runs each of size 5 blocks (except the last run which will have 4 blocks). Hence, after the sort phase, 205 sorted runs are stored as temporary sub-files on disk.

- In the **merging phase,** the sorted runs are merged during one or more **passes.** The **degree of merging** is the **number** of **runs** that can be **merged** together in **each** pass. In each pass, one buffer block is needed to hold one block from each of the runs being **merged**, and one **block** is needed for containing **one block** of the merge result.
  - In our example, = 4 (***four-way merging***), so the 205 initial sorted runs would be merged into 52 at the end of the first pass, which are then merged into 13, then 4, then 1 run, which means that ***four passes*** are needed. The minimum of 2 gives the ***worst-case performance*** of the algorithm, which is:
    - $(2 * b) + (2 * (b * (\log_2 b)))$

- The ***first term represents*** the number of block **accesses** for the ***sort*** phase, since each file block is ***accessed twice*—*once*** for ***reading*** into memory and ***once*** for ***writing*** the records back to disk after sorting.
- The ***second term represents*** the number of block ***accesses*** for the ***merge*** phase, assuming the ***worst-case*** of **2**. In general, the ***log is taken to the base*** and the expression for number of block accesses becomes.

1.3.2 Implementing the SELECT Operation

- There are many options for executing a **SELECT** operation; some depend on the file having specific access paths and may apply only to certain types of selection conditions.
- We discuss some of the *algorithms for implementing SELECT* in this section. We will use the following operations, specified on the relational database, to **illustrate** our discussion:

(OP1): $\sigma_{SSN='123456789'}(EMPLOYEE)$

(OP2): $\sigma_{DNUMBER>5}(DEPARTMENT)$

(OP3): $\sigma_{DNO=5}(EMPLOYEE)$

(OP4): $\sigma_{DNO=5\ AND\ SALARY>30000\ AND\ SEX='F'}(EMPLOYEE)$

(OP5): $\sigma_{ESSN='123456789'\ AND\ PNO=10}(WORKS_ON)$

Search Methods for Simple Selection

- A number of *search algorithms are possible* for selecting records from a file. These are also known as **file scans,** because they scan the records of a file to search for and retrieve records that satisfy a selection condition. If the search algorithm involves the use of an index, the index search is called an **index scan.**
- The following search methods (S1 through S6) are examples of some of the search algorithms that can be used to implement a select operation:

S1. ***Linear search (brute force):***

Retrieve ***every record*** in the file, and test whether its attribute values satisfy the selection condition.

S2. ***Binary search:***

If the selection condition involves an ***equality comparison*** on a ***key attribute*** on which the file is ordered, binary search—which is more efficient than linear search—can be used. An example is OP1 if SSN is the ordering attribute for the EMPLOYEE file.

S3. ***Using a primary index (or hash key):***

If the selection condition involves an ***equality comparison*** on a ***key attribute*** with a ***primary index*** (or ***hash key***)—for example, SSN ='123456789' in OP1—use the primary index (or hash key) to retrieve the record. Note that this condition retrieves a single record (at most).

S4. ***Using a primary index to retrieve multiple records:***

If the comparison condition is >, >=, <, or <= on a key field with a ***primary index***—for example, DNUMBER > 5 in OP2—use the index to find the record satisfying the corresponding equality condition (DNUMBER = 5), then retrieve all subsequent records in the (ordered) file. For the condition DNUMBER < 5, retrieve all the preceding records.

S5. ***Using a clustering index to retrieve multiple records:***

If the selection condition involves an equality comparison on a ***non-key attribute*** with a clustering index—for example, DNO = 5 in OP3—use the index to retrieve all the records satisfying the condition.

S6. ***Using a secondary (-tree) index on an equality comparison:***

This search method can be used to retrieve a single record if the indexing field is a ***key*** (has unique values) or to retrieve multiple records if the indexing field is ***not a key***. This can also be used for ***comparisons*** involving >, >=, <, or <=.

- Method *S1 applies to* ***any file***, but all the *other methods depend on having the appropriate access path on the attribute* used in the selection condition. Methods S4 and S6 can be used to retrieve records in a certain ***range***—for example, 30000<=SALARY<=35000. Queries involving such conditions are called ***range queries***.

1.3.3 Implementing the JOIN Operation

- The ***JOIN*** operation is one of the most time-consuming operations in query processing. Many of the join operations encountered in queries are of the ***EQUIJOIN*** and ***NATURAL JOIN*** varieties, so we consider only these two here. For the remainder of this chapter, the term ***join*** refers to an ***EQUIJOIN*** (or NATURAL JOIN).
- There are many possible ways to implement a ***two-way join***, which is a join on two files. Joins involving ***more than two*** files are called ***multiway joins***. The number of possible ways to execute multiway joins grows very rapidly. In this section we discuss techniques for implementing only two-way joins.
- The algorithms we consider are for join operations of the form:

$$R_{A=B}\ S$$

where ***A*** and ***B*** are ***domain-compatible attributes*** of *R* and *S*, ***respectively***. The methods we discuss can be extended to more general forms of join.

  - We illustrate **four** of the ***most common techniques*** for performing such a join, using the ***following example*** operations:

    (OP6): EMPLOYEE DNO=DNUMBER DEPARTMENT

    (OP7): DEPARTMENT MGRSSN=SSN EMPLOYEE

Methods for Implementing Joins

J1. ***Nested-loop join (brute force):***

For each record *t* in *R* (***outer*** loop), retrieve every record *s* from *S* (***inner*** loop) and test ***whether*** the two records ***satisfy*** the ***join*** condition ***t[A] = s[B].***

- **J2.** *Single-loop join (using an access structure to retrieve the matching records):*

If an *index* (or *hash* key) exists for *one* of the *two join* attributes—say, *B* of *S*—retrieve *each record t* in *R*, one at a time (*single* loop), and *then use the access* structure to retrieve directly *all matching records s* from *S* that satisfy *s*[*B*] = *t*[*A*].

J3. ***Sort–merge join:***

- If the records of ***R*** and ***S*** are ***physically sorted*** (ordered) by value of the join attributes *A* and *B*, respectively, we can implement the join in the ***most efficient*** way possible.
- ***Both files are scanned concurrently*** in order of the join attributes, ***matching*** the records that have the ***same values*** for *A* and *B*. If the files are ***not sorted***, they may be ***sorted first*** by using ***external*** sorting.
  - In this method, ***pairs of file blocks*** are copied into ***memory buffers*** in order and the records of each file are scanned only once each for matching with the other file—***unless both*** *A* and *B* are ***non-key*** attributes, in which case the method needs to be modified slightly.
- The **indexes** provide the ability to access (scan) the records in order of the join attributes, but the records themselves are ***physically scattered*** all over the ***file blocks***, so this method may be quite inefficient, as every record access may involve accessing a different ***disk block***.

J4. ***Hash-join:***

- The ***records of files R and S are both hashed*** to the ***same hash file***, using the same ***hashing function*** on the join attributes *A* of *R* and *B* of *S* as ***hash keys***.
- ***First***, a ***single pass*** through the ***file with fewer records*** (say, *R*) hashes its records to the ***hash*** file ***buckets***;
  - This is called the **partitioning phase,** since the records of *R* are *partitioned* into the *hash buckets*.
- In the ***second phase***, called the **probing phase,** a single pass through the other file (*S*) ***then hashes*** each of its ***records*** to ***probe*** the appropriate ***bucket***, and that record is ***combined*** with ***all matching records*** from *R* in ***that bucket***.

- This simplified description of hash-join assumes that the smaller of the two files *fits entirely into memory buckets* after the first phase. We will discuss variations of hash-join that do not require this assumption below.

1.4 Using Heuristics in Query Optimization

- In this section we will discuss ***optimization techniques*** that apply ***heuristic rules*** to modify the ***internal representation*** of a ***query*** — which is usually in the form of a ***query tree*** or a ***query graph*** data structure — to improve its expected performance.
- The ***parser of a high-level*** query ***first*** generates an ***initial internal representation***, which is then ***optimized*** according to ***heuristic rules***.
  - Following that, a *query execution plan* is generated to execute groups of operations *based on the access paths available* on the files involved in the query.
- One of the main ***heuristic rules*** is to apply ***SELECT*** and ***PROJECT*** operations ***before*** applying the ***JOIN*** or other binary operations.
  - This is ***because the size*** of the ***file resulting*** from a ***binary*** operation — such as ***JOIN*** — is usually a ***multiplicative function*** of the ***sizes*** of the ***input files***.
  - The ***SELECT*** and ***PROJECT*** operations ***reduce the size*** of a file and ***hence*** should be applied ***before*** a ***join*** or ***other binary*** operation.
- ***Query tree*** and ***query graph*** notations can be used *as the basis for the data structures* that are used for *internal representation of queries*.
  - A ***query tree*** is used to represent a ***relational algebra*** or extended relational algebra expression, whereas a ***query graph*** is used to represent a ***relational calculus*** expression.

1.4.1 Notation for Query Trees and Query Graphs

- A ***query tree*** is a ***tree data*** structure that corresponds to a ***relational algebra*** expression. It represents the input relations of the query as ***leaf nodes*** of the tree, and represents the ***relational algebra operations*** as ***internal nodes***.
- An execution of the ***query tree*** consists of executing an ***internal node*** operation whenever its operands are available and then ***replacing*** that ***internal*** node by the ***relation*** that results from ***executing*** the operation.
  - The ***execution terminates*** when the ***root node*** is ***executed*** and produces the ***result relation*** for the query.

Example: $DNUM=DNUMBER(DEPARTMENT))MGRSSN=SSN(EMPLOYEE))$

Corresponds to this query:

```
SELECT P.PNUMBER, P.DNUM, E.LNAME, E.ADDRESS, BDATE
FROM   PROJECT AS P, DEPARTMENT AS D, EMPLOYEE AS E
WHERE  P.DNUM=D.DNUMBER AND D.MGRSSN=E.SSN AND
PLOCATION='MIT';
```

- The three ***relations PROJECT***, ***DEPARTMENT***, and ***EMPLOYEE*** are represented by leaf nodes ***P***, ***D***, and ***E***, while the ***relational algebra*** operations of the expression are represented by ***internal tree nodes***.
- As we can observe, the *query tree represents* a **specific order** of operations for ***executing*** a query. A **more neutral** representation of a query is the **query graph** notation.
  - ***Relations*** in the query are represented by ***relation nodes*****,** which are displayed as *single circles*.
  - *Constant values*, typically from the query *selection* conditions, are represented by ***constant nodes*****,** which are displayed as ***double circles***.
  - ***Selection*** and ***join*** conditions are represented by the ***graph edges***.
  - ***Finally***, the attributes to be retrieved from ***each relation*** are displayed in ***square brackets*** above each relation.
- ***The query graph*** representation ***does not*** indicate ***an order*** on which operations to perform ***first***. There is ***only*** a ***single graph*** corresponding to ***each query***.
- ***Although*** *some* optimization *techniques* were *based on query graphs*, it is now generally *accepted* that *query trees* are ***preferable*** because, in *practice*, the query optimizer needs to show the *order of operations* for query execution, which *is* ***not possible*** *in query graphs*.

1.5 Using Selectivity and Cost Estimates in Query Optimization

- A ***query optimizer*** should ***not depend solely*** on ***heuristic rules***; it should ***also estimate*** and ***compare*** the ***costs*** of ***executing*** a query using different execution strategies and ***should choose*** the strategy with the ***lowest cost estimate***.
  - For this approach to work, accurate *cost estimates* are required so that different strategies are compared *fairly* and *realistically*.
- In addition, we must limit the number of execution strategies to be considered; otherwise, too much time will be spent making cost estimates for the many possible

execution strategies.

- Hence, this approach is ***more suitable*** for ***compiled queries*** where the optimization is ***done*** at ***compile time*** and the resulting execution strategy code is stored and executed ***directly*** at ***runtime***.
- For ***interpreted queries*****,** where the ***entire process*** occurs at ***runtime***, a full-scale optimization may slow down the response time.
  - A ***more elaborate*** optimization is indicated for ***compiled*** queries, ***whereas*** a ***partial***, ***less time-consuming*** optimization works best for ***interpreted*** queries.
  - We call this approach **cost-based query optimization**, and it *uses traditional optimization techniques* that search the *solution space* to a problem for a solution that minimizes an *objective (cost) function*.
  - The ***cost functions*** used in query optimization are estimates and ***not exact cost*** functions, so the optimization may ***select*** a ***query*** execution strategy that is ***not*** the ***optimal*** one.

❑The cost of *executing a query* includes the following components.

1)Access cost to secondary storage:

- This is the ***cost of searching*** for, **reading**, and **writing** data blocks that ***reside on secondary storage***, mainly on **disk**.
- The cost of ***searching for records*** in a file depends on the ***type of access structures*** on that file, such as ***ordering***, ***hashing***, and ***primary or secondary*** indexes.
- In addition, **factors** such as whether the **file blocks** are allocated **contiguously** on the same disk **cylinder** or scattered on the disk affect the **access** cost.

2)Storage cost:

- This is the ***cost of storing*** any ***intermediate files*** that are generated by ***an execution strategy*** for the query.

3) Computation cost :

- This is the **cost** of performing **in-memory operations** on the data ***buffers during query execution***.
- Such operations include **searching** for and **sorting records**, **merging records** for a join, and **performing computations** on field values.

4) Memory usage cost:

- This is the ***cost pertaining*** to the ***number of memory buffers needed*** during ***query execution***.

5) Communication cost:

- This is the ***cost of shipping the query*** and ***its results from the database site*** to the **site or terminal** where the ***query originated***.

1.5.1 Catalog Information Used in Cost Functions

- To ***estimate*** the ***costs*** of ***various execution strategies***, we must keep track of ***any information*** that is needed for the ***cost functions***.
- This ***information*** may be stored in the ***DBMS catalog***, where it ***is accessed*** by the ***query optimizer***.
- **First**, we must know the ***size of each file***.
  - For a file whose records are all of the ***same type***, the ***number of records (tuples) (r)***, the (average) ***record size (R),*** and the ***number of blocks (b)*** (or close estimates of them) are needed.
  - The ***blocking factor (bfr)*** for the file may also be needed.
- We must *also keep track* of the ***primary access method*** and the ***primary access attributes*** for each file.
  - The ***file records*** may be ***unordered***, ***ordered*** by an attribute with or without a ***primary*** or ***clustering*** index, or ***hashed*** on a ***key*** attribute.

- Information is kept on ***all secondary indexes*** and ***indexing attributes***.
- The ***number of levels (x)*** of each ***multilevel index*** (***primary***, ***secondary***, or ***clustering***) is needed for ***cost functions*** that estimate the number of ***block accesses*** that occur ***during query*** execution.
- In some ***cost*** functions the ***number of first-level index blocks*** is needed.

♣ Another important parameter is the ***number of distinct values (d)*** of an attribute and its ***selectivity (sl)***, which is the ***fraction*** of records satisfying an ***equality condition*** on the attribute.

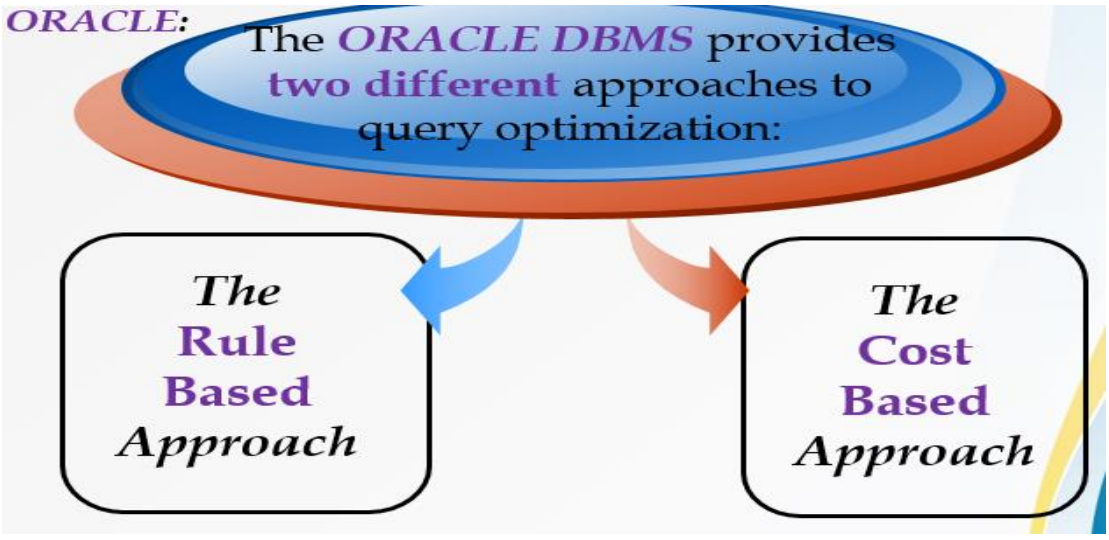
- This allows estimation of the ***selection cardinality*** (***s = sl * r***) of an attribute, which is the ***average*** number of records that will *satisfy an equality selection condition* on that attribute.
  - For a ***key attribute***, ***d = r, sl =1/r*** and hence ***s = sl * r***, since ***sl =1/r*** then ***s = 1/r * r = 1***.
  - For a ***non-key attribute***, by making an assumption that the ***d*** distinct values are ***uniformly distributed*** among the records, we ***estimate sl = (1/d)*** and hence ***s = sl * r***, since ***sl =1/d*** then ***s = 1/d * r = (r/d)***.

♣ **Information** such as the *number of index levels is easy* to maintain because it ***does not change very*** often. **However**, other information may **change frequently**;

- For example, the *number of records* ***r*** in a file changes *every time* a record is ***inserted*** or ***deleted***.

♣ The query optimizer will need reasonably close but not necessarily completely *up-to-the-minute values of these parameters* for use in estimating the ***cost of various execution strategies***.

1.6 Overview of Query Optimization in ORACLE



- With the ***rule-based approach***, the **optimizer** chooses *execution plans* based on ***heuristically ranked operations***.
- ORACLE maintains *a table of ranked access paths*, where a ***lower ranking implies*** a **more efficient** approach.
  - The *access paths range* from table access by *ROWID* (*most efficient*)— where *ROWID* specifies the *record's physical address* that includes:
    1. The **data file**,
    2. **Data block**, and
    3. *Row offset*

    within the block— to a *full table scan* (least efficient)—*where all rows in the table are searched* by doing **multi-block** reads.
- However, the *rule-based approach* is ***being phased out*** in **favor** of the *cost-based approach*, where the *optimizer examines alternative access paths* and *operator algorithms* and chooses the execution plan with ***lowest estimated cost***.
- ***The catalog tables*** containing ***statistical information*** are used in a ***similar fashion***, as described in Section 1.5.
- The **estimated query cost** is *proportional* to the *expected elapsed time* needed to *execute the query* with the *given execution plan*.
  - The ***ORACLE optimizer calculates*** this **cost** *based on the estimated usage of resources*, such as:
    - ✓ ***Input/Output***,
    - ✓ ***CPU time*** and
    - ✓ ***Memory Needed***
- The goal of ***cost-based optimization*** in ***ORACLE*** is to ***minimize the elapsed time*** to process the entire query.
- The ***cost-based optimization*** of ORACLE-8 is a good example of the ***sophisticated approach*** taken to *optimize SQL queries in commercial RDBMSs*.

1.7 Semantic Query Optimization

- A *different approach* to query optimization, called **semantic query optimization**, has been ***suggested***.
- This **technique**, which may be used in *combination with the techniques discussed previously*, ***uses constraints specified on the database schema*** — such as:
  - ***Unique attributes*** and
  - ***Other more complex constraints*** — in order to *modify one query into another query* that is more efficient to execute.
- We will not discuss this approach in detail *but only illustrate it with a simple example*. Consider the *SQL query*:

```
SELECT  E.LNAME, M.LNAME
FROM    EMPLOYEE AS E, EMPLOYEE AS M
WHERE   E.SUPERSSN=M.SSN AND
        E.SALARY.M.SALARY
```

- This query **retrieves** the names of ***employees who earn more than their supervisors***.
  - Suppose that *we had a constraint on the database schema* that stated that **no employee can earn more than his or her direct** supervisor.
- If the semantic query ***optimizer checks for the existence*** of this **constraint**, it ***need not execute the query*** at all because it knows that the result of the **query will be empty**.
  - This may **save considerable time** if the **constraint checking can be done efficiently**.
  - **However**, searching through *many constraints to find those that are applicable to a given query* and that **may semantically optimize it can also be quite time-consuming**.
- With the ***inclusion of active rules*** in database systems ***semantic query optimization techniques*** may ***eventually be fully incorporated*** into the **DBMSs of the future**.

CHAPTER – TWO: TRANSACTION PROCESSING

Transaction Processing

2.1 Introduction to Transaction Processing

- The concept of ***transaction provides*** a mechanism for describing ***logical units*** of database processing.
- **Transaction processing systems** are systems with ***large*** databases and ***hundreds*** of ***concurrent*** users that are executing database transactions.
  - **Examples** of such systems include systems for reservations, banking, credit card processing, stock markets, supermarket checkout, and other similar systems.
- They require ***high availability*** and ***fast response time*** for ***hundreds*** of ***concurrent*** users.
  - In this chapter we present the concepts that are needed in transaction processing systems. We define the concept of a transaction, which is used to represent a logical unit of database processing that must be completed in its entirety to ensure correctness. We discuss the concurrency control problem, which occurs when multiple transactions submitted by various users interfere with one another in a way that produces incorrect results. We also discuss recovery from transaction failures.

2.1.1 Single-User versus Multiuser Systems

- One **criterion** *for classifying* a *database* system is according to the ***number*** of *users* who can use the system **concurrently**—that is, at the same time.
- A DBMS is ***single-user*** if at most ***one user*** at a time can use the system, and it is ***multiuser*** if ***many users*** can use the system—and hence access the database—***concurrently***.
- Single-user DBMSs are mostly restricted to some ***microcomputer*** systems; ***most*** other ***DBMSs*** are multiuser.
  - For example, an airline reservations system is used by hundreds of travel agents and reservation clerks concurrently. Systems in banks, insurance agencies, stock exchanges, supermarkets, and the like are also operated on by many users who submit transactions concurrently to the system.
- **Multiple** users can access databases—and use computer systems—**simultaneously** because of the concept of **multiprogramming**, which allows the computer to execute multiple programs—or ***processes***—at the ***same time***.
- If ***only*** a ***single*** central processing unit (***CPU***) ***exists***, it can actually execute ***at most one*** process ***at a time***.

- ♣ However, **multiprogramming operating systems** execute some commands from one process, then suspend that process and execute some commands from the next process, and so on.

- A **process** is **resumed** at the point where it was suspended whenever it gets its turn to use the **CPU** again.
  - ♣ Hence, concurrent execution of processes is actually **interleaved,** in two processes ***A*** and ***B*** executing ***concurrently*** in an ***interleaved*** fashion.
    - *Interleaving* keeps the *CPU busy* when a process requires an input or output (*I/O*) operation, such as *reading* a *block* from disk.
    - The *CPU* is switched to *execute* another process *rather than remaining idle* during I/O time. Interleaving *also prevents* a long process from *delaying* other processes.
  - ♣ If the computer system has *multiple hardware* processors (CPUs), **parallel processing** of multiple processes is possible.
    - Most of the theory concerning concurrency control in databases is developed in terms of **interleaved concurrency,** so for the remainder of this chapter we assume this model.
    - In a ***multiuser*** DBMS, the ***stored*** data items are the ***primary*** resources that may be accessed ***concurrently*** by interactive users or application programs, which are constantly retrieving information from and modifying the database.

2.1.2 Transactions, Read and Write Operations, and DBMS Buffers

- ♣ A **transaction** is a **logical unit** of database **processing** that includes **one** or **more database** access operations—these can include *insertion*, *deletion*, *modification*, or *retrieval* operations.
- ♣ The database operations that form a transaction can ***either*** be ***embedded*** within an ***application*** program or they can be ***specified interactively*** via a ***high-level*** query language such as ***SQL***.
- ♣ One way of specifying the transaction boundaries is by specifying explicit **begin transaction** and **end transaction** statements in an application program; in this case, all database access operations between the **two** are considered as **forming one** transaction.
- ♣ A single application program may contain *more than one transaction* if it contains *several transaction boundaries*.
  - If the database operations in a transaction **do not update** the database but **only retrieve data**, the transaction is called a **read-only transaction.**
- ♣ The **model** of a database that is used to explain transaction processing concepts is much simplified.
  - ♣ A **database** is basically represented as a collection of **named data items.** The **size** of a data item is called its **granularity,** and it can be a **field** of **some record** in the database, or it may be a **larger unit** such as a record or ***even a whole disk block***, but the ***concepts*** we discuss are ***independent*** of the ***data item granularity***.

- Using ***this simplified database model***, the basic database access operations that a **transaction** can include are as follows:

  1) **read_item(*X*):** Reads a database item named *X* into a program variable. To simplify our notation, we assume that *the program variable is also named X.*
  2) **write_item(*X*):** Writes the value of program variable *X* into the database item named *X.*

- It is obvious that the **basic unit** of ***data transfer*** from **disk** to *main memory* is *one block*. Executing a ***read_item(X)*** command includes the following **steps**:

  1. Find the ***address*** of the ***disk block*** that contains ***item*** *X*.
  2. Copy that disk block ***into*** a ***buffer*** in ***main memory*** (if that disk block is not already in some main memory buffer).
  3. ***Copy*** item ***X from*** the ***buffer*** to the program variable ***named X***.

- Executing a ***write_item(X)*** command includes the following **steps**:

  1. **Find** the address of the ***disk block*** that contains item ***X***.
  2. **Copy** that ***disk block*** into a buffer in ***main memory*** (if that disk block is not already in some main memory buffer).
  3. **Copy item *X*** from the program variable named ***X* into** its correct location in the buffer.
  4. **Store** the **updated** block from the ***buffer back to disk*** (either immediately or at some later point in time).

- *Step 4* is the one that actually *updates* the database on *disk*. In some cases the buffer is *not immediately stored to disk*, in case additional changes are to be made to the *buffer*. Usually, the decision about when to store back a modified disk block that is in a *main memory* buffer is handled by the *recovery manager* of the DBMS in cooperation with the *underlying* operating system.

- The DBMS will generally maintain a number of ***buffers*** in main memory that hold database disk ***blocks*** containing the ***database items*** being processed.
- When these *buffers* are *all occupied*, and additional database *blocks* must be copied *into memory*, some *buffer replacement* policy is used to choose which of the current *buffers* is to be replaced.

- If the ***chosen buffer*** has been ***modified***, it ***must*** be ***written back*** to disk ***before*** it is ***reused***.
  - A transaction includes ***read_item*** and ***write_item*** operations to access and update the database. The **read-set** of a transaction is the set of **all** items that the transaction **reads**, and the **write-set** is the set of **all** items that the transaction **writes**.

2.2 Transaction Processing and System Concepts

2.2.1 Transaction States and Additional Operations

- A **transaction** is an **atomic unit** of **work** that is either **completed** in its **entirety** or **not done** at all.
- For ***recovery purposes***, the system needs to keep track of ***when*** the transaction ***starts***, ***terminates***, and ***commits*** or ***aborts*** (see below).
- Hence, the **recovery manager** keeps track of the following operations:
  1) **BEGIN_TRANSACTION**: This marks the **beginning** of transaction execution.
  2) **READ orWRITE**: These specify **read** or **write** operations on the database items that are **executed** as part of a transaction.
  3) **END_TRANSACTION**: This specifies that **READ** and **WRITE** transaction operations have **ended** and marks the **end** of transaction execution.
     - ***However***, at this point it may be necessary to ***check*** whether the changes introduced by the transaction can be permanently applied to the database (***committed***) or whether the transaction has to be ***aborted*** because it violates *serializability* or for some other reason.
  4) **COMMIT_TRANSACTION**: This signals a ***successful end*** of the transaction so that any **changes** (**updates**) executed by the transaction can be safely **committed** to the database and **will not** be **undone**.
  5) **ROLLBACK (or ABORT)**: This signals that the transaction has ***ended unsuccessfully***, so that **any changes** or effects that the transaction may have applied to the **database** must be ***undone***.
- A **transaction** goes into an **active state** immediately after it ***starts execution***, where it can issue *READ* and *WRITE* operations. When the transaction ends, it **moves** to the **partially committed state.**
  - At **this point**, some recovery **protocols** need to ensure that a *system failure will not result* in an **inability** to record the changes of the transaction **permanently** (usually by recording changes in the **system log**, discussed in the next sub-section).
    - Once this **check** is **successful**, the transaction is said to have **reached** *its commit* point and enters the **committed state.**
    - Once a ***transaction is committed***, it has concluded its execution successfully and ***all its changes*** must be recorded **permanently** in the database.
- **However**, a transaction can go to the **failed state** if one of the **checks** fails or if the transaction is **aborted** during its **active state**.

- The transaction may then have to be *rolled back* to **undo** the effect of its *WRITE* operations on the database.
- The **terminated state** corresponds to the transaction ***leaving*** the system.
- The transaction ***information*** that is maintained in ***system tables*** while the transaction has been **running** is **removed** when the transaction terminates.

♣ **Failed** or ***aborted transactions*** may be ***restarted*** later—either **automatically** or after **being resubmitted** by the user—**as brand new** transactions.

2.2.2 The System Log

♣ To be able to ***recover from failures*** that affect transactions, the *system* maintains a **log** to *keep track* of all transaction operations that affect the values of database items.
- This **information** may be needed to permit **recovery** *from failures*. The **log** is **kept** on disk, so it is not affected by any type of failure except for disk or ***catastrophic*** failure.
- In addition, the **log** is periodically ***backed up*** to *archival* storage (**tape**) to guard against such ***catastrophic*** failures.

♣ We **now list** the types of entries—called **log records**—that are *written to the log* and *the action each performs*. In these entries, *T refers* to a unique **transaction-id** that is generated automatically by the system and is used to identify each transaction:
1) **[start_transaction, *T*]**: Indicates that transaction *T* has started execution.
2) **[write_item, *T,X,old_value,new_value*]:** Indicates that transaction *T* has changed the value of database item *X* from ***old_value*** to ***new_value***.
3) **[read_item, *T, X*]**: Indicates that transaction ***T has*** read the value of database item *X*.
4) **[commit,*T*]**: Indicates that transaction *T* has ***completed successfully***, and affirms that its effect can be committed (*recorded permanently*) to the database.
5) **[abort,*T*]**: Indicates that transaction ***T* has been aborted**.

♣ **Notice** that we assume here that ***all permanent*** changes to the database occur within transactions, so the *notion of recovery* from a ***transaction failure*** amounts to either **undoing** or **redoing** transaction operations individually ***from the log***.

♣ If the system **crashes**, we can **recover** to a consistent database state by **examining** the **log**.
- **Because** the **log** contains a record of **every WRITE** operation that changes the value of some database item, it is possible to **undo** the effect of these **WRITE** operations of a transaction *T* by tracing **backward** through the log and **resetting** all items changed by a **WRITE** operation of *T* to their **old_values**.

♣ **Redoing** the operations of a transaction may also be needed if all its **updates** are **recorded** in the **log** but a **failure** occurs **before** we can be **sure** that all these **new_values** have been written **permanently** in the actual database on disk.

♣ Redoing the operations of *transaction T* is applied by **tracing** forward through the **log** and **setting** all items changed by a WRITE operation of T to their **new_values**.

2.3 Desirable Properties of Transactions

- ***Transactions*** should possess several properties. These are often called the **ACID properties,** and they should be *enforced* by the *concurrency control* and *recovery* methods of the *DBMS*. The following are the ACID properties:
  1. **Atomicity:** A transaction is an *atomic unit* of processing; it is either performed in its ***entirety*** or ***not performed*** at all.
  2. **Consistency preservation:** A transaction is consistency preserving if its *complete* execution take(s) the database from *one* consistent *state* to *another*.
  3. **Isolation:** A transaction should appear as though it is being executed in ***isolation*** from other ***transactions***. That is, the execution of a transaction should not be *interfered* with by *any other* transactions *executing concurrently*.
  4. **Durability or permanency:** The *changes* applied to the database by a *committed* transaction *must persist* in the database. These changes *must not be lost* because of *any failure*.
- The **atomicity** property requires that we execute a transaction to completion.
  - It is the **responsibility** of the *transaction recovery subsystem* of a DBMS to ensure ***atomicity***.
  - If a *transaction fails* to complete for some reason, such as a system crash in the *midst* of *transaction execution*, the recovery technique must ***undo*** any effects of the transaction on the database.
- The **preservation** of consistency is generally considered to be the **responsibility** of the **programmers** who **write** the **database** programs or of the **DBMS module** that **enforces integrity** constraints.
  - Recall that a **database state** is a collection of *all the stored data items* (**values**) in the database at a given ***point in time***.
  - A **consistent state** of the database *satisfies* the constraints specified in the **schema** as well as any other ***constraints*** that should hold on the database.
  - A *database program* should be written in a way that **guarantees** that, if the database is in a **consistent** state before executing the transaction, it will be in a consistent state after the ***complete*** execution of the transaction, assuming that ***no interference with other transactions*** occurs.
- **Isolation** is enforced by the ***concurrency control subsystem*** of the DBMS.
  - If every transaction ***does not*** make its ***updates visible*** to other transactions until it is ***committed***, one form of **isolation** is **enforced** that solves the *temporary update problem* and *eliminates cascading rollbacks*.
  - There have been attempts to define the ***level of isolation*** of a transaction.

- A transaction is said to ***have level 0 (zero) isolation*** if it **does not** overwrite the **dirty reads** of higher-level transactions.
- ***A level 1 (one)****-isolation* transaction *has no lost updates*; and
- ***Level 2****-isolation* has *no lost updates* and *no dirty reads*.
- Finally, ***level 3 isolation*** (also called ***true*** *isolation*) has, in addition to *degree 2 properties*, **repeatable** reads.

♣ **Finally**, the *durability property* is the **responsibility** of the ***recovery subsystem*** of the DBMS.

Details of ACID properties with Examples:

- Transaction to transfer $50 from account A to account B:
  1. **read** (A)
  2. $A := A - 50$
  3. **write**(A)
  4. **read**(B)
  5. $B := B + 50$
  6. **write**(B)

- **Atomicity requirement**
  - If the transaction fails after step 3 and before step 6, money will be "**lost**" leading to an **inconsistent** database state
    - Failure could be due to software or hardware
  - The ***system should ensure*** that updates of a partially executed transaction are not reflected in the database

- **Durability requirement** — once the user has been notified that the *transaction has completed* (i.e., the transfer of the $50 has taken place), the ***updates*** to the database by the transaction ***must persist*** even if there are ***software*** or ***hardware failures***.

- Transaction to transfer $50 from account A to account B:
  1. **read**(A)
  2. $A := A - 50$
  3. **write**(A)
  4. **read**(B)
  5. $B := B + 50$

6. **write**(*B*)

- **Consistency requirement** in above example:
  - The sum of A and B is **unchanged** by the **execution** of the transaction
- In general, consistency requirements include
  - ✓ **Explicitly** specified integrity constraints such as primary keys and foreign keys
  - ✓ **Implicit** integrity constraints
    - e.g. sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
  - A transaction must see a consistent database.
  - During transaction execution the database may be temporarily inconsistent.
  - **When** the transaction completes successfully the database must be consistent
    - ✓ Erroneous transaction logic can lead to inconsistency

- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1	T2
1. **read**(A)	
2. $A := A - 50$	
3. **write**(A)	
	read(A), read(B), print(A+B)
4. **read**(B)	
5. $B := B + 50$	
6. **write**(B	

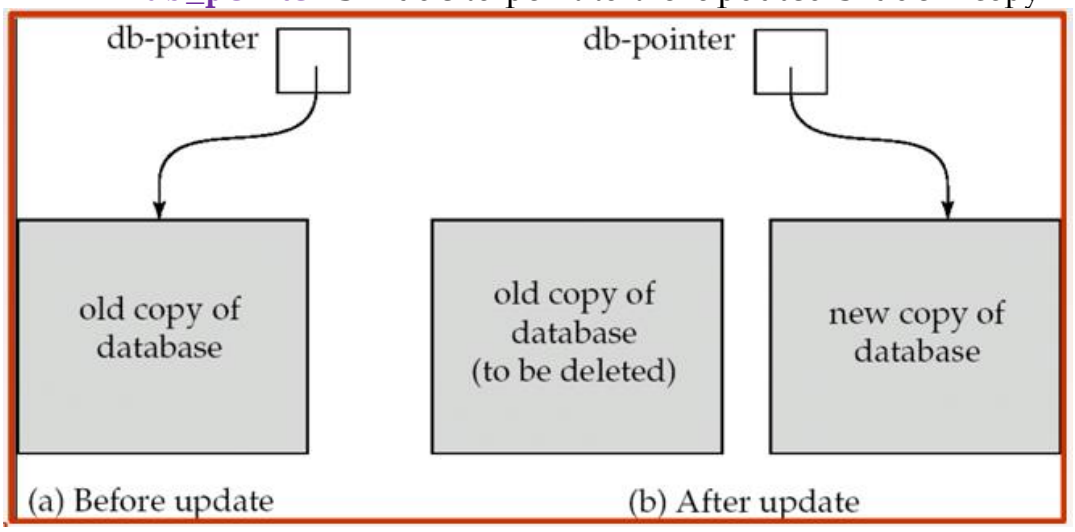
- Isolation can be ensured trivially by running transactions **serially**
  - that is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.

Implementation of Atomicity and Durability

The **recovery-management** component of a database system implements the support for atomicity and durability.

E.g. The ***shadow-database*** scheme:

- All updates are made on a ***shadow copy*** of the database
  - **db_pointer** is made to point to the updated shadow copy



- **db_pointer** *always* points to the ***current consistent*** copy of the database.
  - In case transaction **fails**, *old consistent* copy pointed to by **db_pointer** can be used, and the **shadow copy** can be **deleted**.
- The ***shadow-database*** scheme:
  - ***Assumes*** that **only one** transaction *is active at a time*.
  - ***Assumes*** disks ***do not fail***
  - Useful for ***text editors***, but
    - **extremely** inefficient for large databases
      - Variant *called shadow paging reduces* copying of data, but is still *not practical for large* databases
  - ***Does not handle*** concurrent transactions

2.4 Schedules Based on Recoverability and Serializability

2.4.1 Schedules (Histories) of Transactions

- **Schedule** – a sequences of instructions that specify the ***chronological order*** in which instructions of ***concurrent transactions*** are executed.
  - A **schedule** for a set of transactions ***must consist of all instructions*** of those transactions
  - ***Must preserve the order*** in which the instructions appear in each individual transaction.
- A **schedule** (or **history**) *S* of *n* transactions , , ..., is an ***ordering*** of the operations of the transactions subject to the constraint that, for each transaction that participates in *S*, the operations of in *S* must appear in the same order in which they occur in.
  - Note, ***however***, that operations from other transactions can be ***interleaved*** with the operations of in *S*. For now, consider the order of operations in *S* to be a ***total ordering***, although it is possible theoretically to deal with schedules whose operations form *partial orders* (as we discuss later).
- For the purpose of *recovery* and *concurrency* control, we are mainly interested in the **read_item** and **write_item** operations of the transactions, as well as the commit and abort operations.
  - A shorthand notation for describing a schedule uses the ***symbols r, w, c,*** and ***a*** for the operations ***read_item***, ***write_item***, ***commit***, and ***abort***, ***respectively***, and ***appends*** as subscript the transaction id (transaction number) to each operation in the schedule.

- In this notation, the database item *X* that is **read** or **written** follows the *r* and *w* operations in ***parentheses***.

2.4.2 Schedules Based on Recoverability and Serializability

- For some ***schedules*** it is easy to recover from *transaction failures*, whereas for other schedules the recovery process can be quite involved. Hence, it is important to characterize the types of schedules for which ***recovery is possible***, as well as those for which recovery is ***relatively simple***.
  - These characterizations ***do not actually*** provide the ***recovery algorithm*** but instead ***only attempt*** to theoretically characterize the different types of ***schedules***.
- **First**, we would like to ensure that, ***once*** a transaction T is ***committed***, it should **never** be necessary to ***roll back*** T.
  - The schedules that theoretically meet this criterion are ***called recoverable schedules*** and those that ***do not*** are called ***non-recoverable***, and hence should *not be permitted*.
  - A ***schedule S*** is **recoverable** if *no transaction* T in S commits until all transactions T that have written an item that *T reads have* committed. A transaction T reads from transaction T in a schedule S if some item X is first written by T and later read by T.
  - In addition, *T should not have been aborted before T* reads item X, and there ***should be no*** transactions that **write X** *after T writes it and before T reads* it (unless those transactions, if any, have aborted before T reads X).
- Recoverable schedules require a ***complex recovery*** process, but ***if sufficient information is kept*** (in the log), a *recovery algorithm* can be devised.
- In a recoverable schedule, ***no committed transaction*** ever needs to be **rolled back**.
  - However, it is possible for a phenomenon known as **cascading rollback** (or **cascading abort**) to occur, where an ***uncommitted*** transaction has to be rolled back because it read an item from a transaction that failed. This is illustrated in schedule, where transaction has to be rolled back because it read item *X* from, and then aborted.
  - Cascading rollback can be ***quite time-consuming***—since numerous transactions can be ***rolled back***—it is important to characterize the schedules where this phenomenon is guaranteed *not to occur*.
- A schedule is said to be **cascadeless**, or **avoid cascading rollback**, if every transaction in the schedule *reads only items* that were written by ***committed*** transactions.
  - ***In this case***, all items read *will not be discarded*, so *no cascading rollback* will occur. To satisfy this criterion, the (*X*) command in schedule must be ***postponed*** until after has committed (or aborted), thus *delaying but ensuring no cascading* rollback if aborts.

- Finally, there is a **third**, more **restrictive** type of schedule, called a **strict schedule**, in which transactions can *neither read nor write* an item *X* until the last transaction that *wrote X has committed* (or aborted).
  - Strict schedules ***simplify*** the ***recovery*** process. In a strict schedule, the process of undoing a ***write_item***(*X*) operation of an aborted transaction is simply to restore the ***before image*** (*old_value*) of data item *X*.
  - This simple procedure ***always works correctly*** for **strict schedules**, but it may **not work** for ***recoverable*** or ***cascadeless*** schedules.
- Now let's look the Schedules based on Serializability by considering different schedules.
- The basic Assumption in **Serializability** is that **each transaction preserves** database **consistency**.
  - Thus **serial execution** of a set of transactions **preserves database consistency.**

Schedule 1

- Let T_1 transfer \$50 from *A* to *B*, and T_2 transfer 10% of the balance from *A* to *B*.
- A serial schedule in which T_1 is followed by T_2 :

T_1	T_2
read(A)	
$A := A - 50$	
write (A)	
read(B)	
$B := B + 50$	
write(B)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)

Schedule 2

- A serial schedule where T_2 is followed by T_1

T_1	T_2
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)
read(A)	
$A := A - 50$	
write(A)	
read(B)	
$B := B + 50$	
write(B)	

Schedule 3

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1.

T_1	T_2
read(A)	
$A := A - 50$	
write(A)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
read(B)	
$B := B + 50$	
write(B)	
	read(B)
	$B := B + temp$
	write(B)

In Schedules 1, 2 and 3, the sum A + B is preserved.

Schedule 4

- The following concurrent schedule does not preserve the value of $(A + B)$.

T_1	T_2
read(A)	
$A := A - 50$	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
write(A)	
read(B)	
$B := B + 50$	
write(B)	
	$B := B + temp$
	write(B)

2.5 Transaction Support in SQL

- The definition of an ***SQL-transaction*** is similar to our ***already defined concept*** of a transaction. That is, it is a ***logical unit of work*** and is ***guaranteed to be atomic***.
  - A ***single SQL statement*** is **always** considered to be **atomic**—***either*** it ***completes*** execution ***without error*** or it **fails** and leaves the ***database unchanged***.
- With SQL, there is **no** explicit **Begin_Transaction** statement. Transaction **initiation** is done **implicitly** when particular SQL statements are encountered.
  - However, every transaction must have ***an explicit end statement***, which is either a ***COMMIT*** or a ***ROLLBACK***.
  - Every transaction has certain **characteristics** attributed to it. These characteristics are specified by a *SET TRANSACTION* statement in SQL.
  - The characteristics are:

1) The ***access mode***,
2) The ***diagnostic area size***, and
3) The ***isolation level***.

- The **access mode** can be specified as ***READ ONLY*** or ***READ WRITE***. The default is *READ WRITE*, unless the **isolation** level of *READ UNCOMMITTED* is specified, in which case *READ ONLY* is assumed.
  - A mode of *READ WRITE* allows ***update***, ***insert***, ***delete*** and ***create*** commands to be executed.
  - A mode of READ ONLY, as the name implies, is simply **for data retrieval**.
- The **diagnostic area size** option, *DIAGNOSTIC SIZE n*, specifies an integer value *n*, indicating the number of conditions that can be held **simultaneously** in the ***diagnostic area***.
  - These **conditions** supply **feedback** *information* (***errors*** or ***exceptions***) to the user on the most recently executed SQL statement.
- The **isolation level** option is specified using the statement *ISOLATION LEVEL* <**isolation**>, where the value for <*isolation*>can be:
  - *READ UNCOMMITTED*,
  - *READ COMMITTED*,
  - *REPEATABLE READ*, or
  - *SERIALIZABLE*.
- The default isolation level is *SERIALIZABLE*, although ***some systems*** use as ***READ COMMITTED*** their default.
  - The use of the term **SERIALIZABLE** here is based on not allowing violations that ***cause dirty read***, **unrepeatable read**, and **phantoms**.
- If a transaction executes at a **lower isolation** level than **SERIALIZABLE**, then one or more of the following ***three violations*** may occur:
  1. **Dirty read:** A transaction may read the update of a transaction, which *has not yet committed*. If ***fails*** and is ***aborted***, then would have read a value that ***does not exist*** and is ***incorrect***.
  2. **Non-repeatable read:** A *transaction* may *read* a *given value* from a table. If another *transaction later updates* that *value* and *reads* that value again, will see a *different* value.
  3. **Phantoms:** A *transaction* may read a *set of rows* from a table, perhaps based on some condition specified in the SQL **WHERE-clause**. Now *suppose* that a *transaction inserts* a ***new row*** that also **satisfies** the *WHERE-clause* condition used in, into the table used by. If is **repeated**, then will **see a phantom**, a row that previously ***did not*** exist.

- Possible Violations Based on Isolation Levels as Defined in SQL:

	Type of Violation		
Isolation level	Dirty read	Nonrepeatable read	Phantom
READ UNCOMMITTED	yes	yes	yes
READ COMMITTED	no	yes	yes
REPEATABLE READ	no	no	yes
SERIALIZABLE	no	no	no

- A sample **SQL transaction** might look like the following:

```
EXEC SQL WHENEVER SQLERROR GOTO UNDO;

EXEC SQL SET TRANSACTION

READ WRITE

DIAGNOSTICS SIZE 5

ISOLATION LEVEL SERIALIZABLE;

EXEC SQL INSERT INTO EMPLOYEE (FNAME, LNAME, SSN, DNO, SALARY)

VALUES ('Robert', 'Smith', '991004321', 2, 35000);

EXEC SQL UPDATE EMPLOYEE
SET SALARY = SALARY * 1.1 WHERE DNO = 2;

EXEC SQL COMMIT;

GOTO THE_END;

UNDO: EXEC SQL ROLLBACK;

THE_END: ...;
```

- The above transaction consists of ***first inserting a new row*** in the **EMPLOYEE** table and then ***updating the salary*** of all employees who work in department 2.
  - If ~~***an error occurs*** on any of the ***SQL statements***, the entire transaction is~~

rolled back. This implies that any updated salary (by this transaction) would be restored to its previous value and that the newly inserted row would be removed.

- As we have seen, SQL provides a number of ***transaction-oriented*** features. The **DBA** or ***database programmers*** can take advantage of these options to try improving transaction performance by ***relaxing Serializability*** if that is **acceptable** for their applications.

CHAPTER – THREE: CONCURRENCY CONTROL TECHNIQUES

Concurrency Control Techniques

3.1 Locking Techniques for Concurrency Control

- Some of the **main techniques** used to control ***concurrent execution*** of transactions are based on the concept of locking data items.
- A **lock** is a variable associated with a data item that describes the status of the item with respect to possible operations that can be applied to it.
- **Generally**, there is ***one lock*** for each data item in the database. **Locks** are used as a means of ***synchronizing*** the access by concurrent transactions to the database items.

Types of Locks and System Lock Tables:

- **Several** types of locks are used in concurrency control. To introduce locking concepts gradually:
  - We **first** discuss ***binary locks***, which are *simple* but *restrictive* and so are *not used* in *practice*.
  - We then discuss ***shared/exclusive locks***, which provide more *general locking capabilities* and are used in ***practical database*** locking schemes.

Binary Locks

- A **binary lock** can have two **states** or **values**:
  - **Locked** and **unlocked** (or 1 and 0, for ***simplicity***).

- A ***distinct lock*** is associated with each database item *X*.
  - If the value of the ***lock*** on ***X is 1***, item ***X cannot be accessed*** by a database operation that requests the item.
  - If the value of the ***lock on X is 0***, the item can be ***accessed*** when ***requested***. We refer to the current value (or state) of the lock associated with item *X* as **LOCK(*X*).**
- ***Two*** operations, ***lock_item***and ***unlock_item***, are used with ***binary locking***.
- A transaction requests access to an item *X* by first issuing a **lock_item(*X*)** operation.
  - If *LOCK(X) = 1*, the transaction is forced to *wait*.
  - If *LOCK(X) = 0*, it is set to 1 (the transaction **locks** the item) and the transaction is *allowed* to *access* item *X*.
- When the transaction is through using the item, it issues an **unlock_item(*X*)** operation, which sets *LOCK(X) to 0* (**unlocks** the item) so that *X* may be accessed by *other transactions*.
- Notice that the ***lock_item*** and ***unlock_item*** operations must be implemented as ***indivisible*** units (known as **critical sections** in operating systems); that is, ***no interleaving*** should be allowed *once* a *lock* or *unlock* operation is started until the operation terminates or the transaction waits.
- Notice that it is ***quite simple*** to implement a **binary lock**; all that is needed is a ***binary-valued*** variable, LOCK, associated with each data item *X* in the database. In its simplest form, each lock can be a record with **three** fields:
  - <Data *item name*, *LOCK*, locking *transaction*> plus a **queue** for transactions that are waiting to access the item.
- The system needs to maintain only these records for the items that are currently locked in a **lock table,** which could be organized ***as a hash file***.
  - Items *not in the lock table* are considered to be **unlocked**.
  - The DBMS has a **lock manager subsystem** to keep track of and control access to locks.
- If the ***simple binary locking*** scheme described here is used, every transaction **must obey** the following rules:

  1. A transaction T must issue the operation ***lock_item(X)*** **before** any **read_item(*X*)** or **write_item(*X*)** operations are performed in T.
  2) A transaction T must issue the operation ***unlock_item***(*X*) after all ***read_item***(*X*) and ***write_item***(*X*) operations are completed in T.
  3) A transaction T will not issue a ***lock_item(X)*** operation if it already holds the ***lock*** on item *X*.
  4) A transaction T will not issue an ***unlock_item***(*X*) operation ***unless*** it already holds the ***lock*** on item *X*.
- These rules can be **enforced** by the ***lock manager module*** of the DBMS.
  - Between the ***lock_item(X)*** and ***unlock_item(X)*** operations in transaction T, T is said to **hold the lock** on item *X*.
- At *most one transaction can hold* the lock on a particular item.
  - Thus *no two transactions can access* the **same** item **concurrently**.

Shared/Exclusive (or Read/Write) Locks

- The preceding **binary** locking scheme is **too restrictive** for database items, **because** at most **one transaction** can hold a lock on a **given** item.
- We should allow several transactions to access the same item *X* if they all access *X* for ***reading purposes only***.
  - However, if a transaction is to **write** an item *X*, it must have **exclusive** access to *X*. For this purpose, a different type of lock called a **multiple-mode lock** is used.
- In this scheme—called **shared/exclusive** or **read/write locks**—there are **three** locking operations:
  1) read_lock(*X*),
  2) write_lock(*X*), and
  3) unlock(*X*).
- A lock associated with an item *X*, **LOCK(*X*)**, now has **three** possible **states**:
  - "read-locked,"
  - "write-locked," or
  - "unlocked"
- A **read-locked item** is also called **share-locked,** because other transactions are allowed to **read** the item, whereas a **write-locked item** is called **exclusive-locked,** because a *single transaction exclusively* holds the **lock** on the item.
- One method for implementing the preceding ***three operations*** on a read/write lock is to *keep track* of the *number of transactions* that hold a ***shared (read)*** lock on an item in the lock table.
- Each record in the lock table will have four fields:
  - <Data **item name**, **LOCK**, **No_of_reads**, **locking_transaction**(s)>.
  - Again, to save space, the system need maintain lock records only for locked items in the lock table.
- The ***value (state)*** of *LOCK* is either *read-locked* or *write-locked*, suitably coded (if we assume no records are kept in the lock table for unlocked items).
  - If LOCK(*X*)=**write-locked**, the value of locking_transaction(s) is a **single** transaction that holds the **exclusive** (**write**) lock on *X*.
  - If LOCK(*X*)=**read-locked**, the value of locking transaction(s) is a list of **one or more** transactions that hold the **shared** (**read**) lock on *X*.
- As before, each of the **three** operations should be considered **indivisible**; no **interleaving** should be allowed once one of the operations is started until either the operation terminates by granting the lock or the transaction is placed on a waiting queue for the item.
  - When we use the shared/exclusive locking scheme, the system **must enforce** the following rules:

1. *A transaction **T must issue** the operation **read_lock**(X) or write_lock(X) before any **read_item(X)** operation is performed in T.*
2. *A transaction T must issue the operation **write_lock**(X) before any **write_item**(X) Operation is performed in T.*
3. *A transaction T must issue the operation **unlock(X)** after all **read_item(X)** and **write_item(X)** operations are completed in T.*
4. *A transaction T **will not issue** a **read_lock(X)** operation if it already holds a read (**shared**) lock*

*or a write (**exclusive**) lock on item X.*
*5. A transaction T **will not issue** a write_lock(X) operation if it already holds a read (shared) lock or write (exclusive) lock on item X.*
*6. A transaction T will **not issue an unlock(X)** operation **unless** it **already** holds a read (shared) lock or a write (exclusive) lock on item X.*

3.2 Concurrency Control Based on Timestamp Ordering

- Recall that a **timestamp** is a unique identifier created by the DBMS to identify a transaction.
  - Typically, ***timestamp values*** are assigned in the **order** in which the transactions are submitted to the system, so a timestamp can be thought of as the *transaction start time*. We will refer to the timestamp of transaction T as **TS(T).**
- ***Concurrency control techniques*** based on timestamp ordering ***do not use locks***; hence, **deadlocks** cannot occur.
- Timestamps can be generated in **several** ways.
  - ***One possibility*** is to **use** a ***counter*** that is incremented each time its value is assigned to a transaction. The transaction **timestamps** are numbered 1, 2, and 3 . . . in this scheme. A computer counter has a finite maximum value, so the system must periodically reset the counter to zero when no transactions are executing for some short period of time.
  - **Another** way to implement timestamps is to **use** the ***current date/time*** value of the system clock and ensure that **no two timestamp** values are generated during the same tick of the clock.

3.2.1 The Timestamp Ordering Algorithms

- The idea for this scheme is to **order** the transactions ***based on their timestamps***. A schedule in which the transactions participate is then **serializable**, and the equivalent serial schedule has the transactions in order of their timestamp values.
  - This is called **timestamp ordering (TO).**
- In timestamp ordering, **however**, the schedule is equivalent to the ***particular serial order*** corresponding to the order of the transaction timestamps. The algorithm must ensure that, for each item accessed by ***conflicting operations*** in the schedule, the order in which the item is accessed ***does not violate*** the **serializability** order.
- To do this, the algorithm associates with each database item *X* ***two timestamp*** (**TS**) values:
  1) **Read_TS(*X*):** The **read timestamp** of item *X*; this is the ***largest timestamp*** among **all** the timestamps of transactions that have successfully ***read item X***—that is, ***read_TS(X)*** = ***TS(T)***, where T is the ***youngest*** transaction that ***has read X successfully***.

2) **Write_TS(*X*):** The **write timestamp** of item *X*; this is the **largest** of all the timestamps of transactions that have successfully written item *X*—that is, write_TS(*X*) = TS(T), where T is the ***youngest*** transaction that has ***written X successfully***.

Basic Timestamp Ordering

- **Whenever** some transaction T tries to issue a ***read_item(X)*** or a ***write_item(X)*** operation, the **basic TO** algorithm compares the **timestamp** of T with **read_TS(*X*)** and **write_TS(*X*)** to ensure that the timestamp **order** of transaction execution **is not violated**.
  - If this ***order is violated***, then transaction T is aborted and resubmitted to the system as a new transaction with a ***new timestamp***.
  - If T is *aborted and rolled* back, any transaction T_1 that may have used a value written by T *must also be rolled back*.
- Similarly, any transaction T_2 that may have used a value written by T_1 must also be ***rolled back***, and so on.
  - This effect is known as **cascading rollback** and is one of the ***problems associated with basic TO***, since the schedules produced are **not recoverable**.
- An ***additional protocol*** must be enforced to ensure that the schedules are **recoverable**, **cascadeless**, or **strict**. We first describe the basic TO algorithm here. The *concurrency control algorithm must check* whether ***conflicting operations*** violate the timestamp ordering in the following **two cases**.

1. Transaction T issues a ***write_item(X)*** operation:
   a. If ***read_TS(X) > TS(T)*** or if ***write_TS(X) > TS(T)***, then ***abort*** and ***roll back*** T and ***reject*** the operation. This should be done because some younger transaction with a timestamp greater than TS(T)—and hence ***after*** **T** in the ***timestamp ordering***—has already read or written the value of item *X* before T had a chance to write *X*, thus violating the timestamp ordering.
   b. If the condition in ***part (a) does not occur***, then **execute** the write_item(*X*) Operation of T and set write_TS(*X*) to TS(T).

2. Transaction T issues a **read_item(*X*)** operation:
   a. If ***read_TS(X) > TS (T)***, then abort and roll back T and reject the operation. This should be done because some younger transaction with timestamp greater than TS(T)—and hence ***after*** **T** in the timestamp ordering—has already read the value of ***item X*** before T had a chance to read *X*.
   b. If read_TS(*X*) <= TS(T), then execute the **read_item**(*X*) operation of T and set read_TS(*X*) to the *larger* of TS(T) and the current read_TS(*X*).

- Hence, whenever the basic TO algorithm **detects two *conflicting operations*** that occur in the **incorrect** order, it **rejects** the later of the **two operations** by aborting the transaction that issued it.
- However, some schedules are possible under **each protocol** that are **not allowed** under the other. Hence, **neither** protocol allows ***all possible* serializable** schedules. As mentioned earlier, deadlock does not occur with timestamp ordering. However, **cyclic restart** (and hence starvation) may occur if a transaction is **continually aborted** and **restarted**.

Strict Timestamp Ordering

- A **variation** of basic **TO** called **strict TO** ensures that the schedules are both **strict** (for easy recoverability) and ***(conflict) serializable***.
- In this variation, a transaction T that issues a read_item(*X*) or write_item(*X*) such that TS(T) > write_TS(*X*) has its read or write operation ***delayed*** until the transaction T that ***wrote*** the value of *X* (hence TS(T) = write_TS(*X*)) has committed or aborted.
- To implement this algorithm, it is necessary to simulate the locking of an item *X* that has been written by transaction T until T is either committed or aborted.
- This algorithm does not cause deadlock, since T waits for T only if TS(T) > TS(T).

Thomas's Write Rule

- A modification of the basic TO algorithm, known as **Thomas's write rule,** *does not enforce* conflict serializability; But it ***rejects fewer write operations***, by modifying the checks for the **write_item(*X*)** operation as follows:
  - If ***read_TS(X) > TS(T)***, then **abort** and **roll back** T and reject the operation.
  - If write_TS(X) > TS(T), then ***do not execute the write operation*** but continue processing.
    - This is **because** some transaction with **timestamp greater** than TS(T)—and hence after T in the timestamp ordering—has **already written** the value of X.
    - Hence, we must **ignore** the **write_item(X)** operation of T because it is already **outdated** and **obsolete**.
    - **Notice** that ***any conflict arising*** from this situation would be detected **by case (1).**
  - If neither the condition in **part (1) nor** the condition in ***part (2) occurs***, then **execute** the write_item(X) operation of T and set write_TS(X) to TS(T).

3.3 Multiversion Concurrency Control Techniques

- Other protocols for concurrency control keep the **old values** of a data item when the item is **updated**.
  - These are **known as multiversion concurrency control,** because several **versions** (**values**) of an item are **maintained**.
- When a transaction requires **access** to an item, an ***appropriate version is chosen*** to maintain the ***serializability*** of the ***currently executing schedule***, if possible.
  - The idea is that ***some read operations*** that would be rejected in other techniques can still be **accepted** by **reading** an ***older version*** of the item to maintain **serializability**.
- When a transaction **writes** an **item**, it **writes** a ***new version*** and the **old** version of the item is **retained**. Some multiversion concurrency control algorithms use the concept of ***view*** **serializability** rather than ***conflict*** **serializability**.
- An **obvious drawback** of **multiversion** techniques is that **more storage** is needed to maintain **multiple versions** of the database items.
- However, **older** versions may have to be **maintained** anyway—for example, for **recovery purposes**.
- In addition, **some database** applications require **older versions** to be kept to maintain a **history** of the **evolution** of data item values.
- The extreme case is a ***temporal database*** , which *keeps track of* all changes and the times at which they occurred.
  - In such cases, there is ***no additional storage penalty*** for ***multiversion*** techniques, since ***older versions are already*** maintained.
- Several ***multiversion concurrency control schemes*** have been proposed. We discuss only one schemes here, **based on timestamp** ordering.

3.3.1 Multiversion Technique Based on Timestamp Ordering

- In this method, **several versions**... of each data item *X* are maintained. For *each version,* the value of version and the following two timestamps are kept:
  1. **read_TS:** The ***read timestamp*** of is the ***largest*** of all the timestamps of transactions that have ***successfully read*** version .
  2. **write_TS:** The ***write timestamp*** of is the timestamp of the transaction that **wrote** the value of **version** .
- Whenever a transaction T is allowed to execute a **write_item(*X*)** operation, a **new version** of item *X* is **created**, with **both** the **write_TS** and the **read_TS** set to **TS(T)**.

- **Correspondingly**, when a transaction T is allowed to **read** the value of **version** X_i, the value of **read_TS()** is set to the larger of the current **read_TS()** and **TS(T)**.

3.4 Validation (Optimistic) Concurrency Control Techniques

- In ***all the concurrency control techniques*** we have discussed **so far**, a **certain** degree of **checking** is **done** *before* a database operation can be executed.
  - For example, in **locking**, a **check** is done to determine whether the item being **accessed** is **locked**.
  - In timestamp ordering, the transaction timestamp is checked against the read and write timestamps of the item.
  - Such **checking** represents **overhead** during **transaction execution**, with the effect of **slowing down** the transactions.
- In **optimistic concurrency control techniques**, also known as **validation** or **certification techniques**, ***no checking*** is done **while** the transaction is **executing**.
- Several proposed concurrency control methods use the **validation technique**. We will describe ***only one scheme*** here.
  - In this scheme, **updates** in the transaction are ***not* applied directly** to the database items ***until the transaction reaches its end***.
  - During transaction execution, **all updates** are applied to ***local copies*** of the data items that are ***kept for the transaction***.
  - At the **end** of transaction execution, a **validation phase** checks whether any of the transaction's updates **violate serializability**.
- Certain information needed by the ***validation phase must*** be kept by the system. If **serializability** is ***not violated***, the transaction is **committed** and the database is updated from the **local copies**; **otherwise**, the transaction is **aborted** and then **restarted** later.
- There are **three phases** for this ***concurrency control protocol***:
  1. **Read phase:** A transaction can read values of committed data items from the database. However, **updates** are **applied only** to **local** copies (versions) of the data items kept in the transaction workspace.
  2. **Validation phase:** Checking is performed to ensure that *serializability will not* be violated *if the transaction updates are applied* to the database.
  3. **Write phase:** If the **validation** phase is **successful**, the **transaction updates** are applied to the database; **otherwise**, the **updates** are **discarded** and the **transaction** is restarted.

- The idea **behind *optimistic concurrency control*** is to do all the **checks at once**; hence, transaction execution proceeds with a **minimum of overhead** until the **validation** phase is reached.
- If there is ***little interference*** among transactions, *most will be validated successfully*.
  - However, if there is **much interference**, many transactions that ***execute*** to

completion will have their ***results discarded*** and ***must*** be ***restarted*** later.

- Under these **circumstances**, optimistic techniques ***do not work well***. The techniques are called "**optimistic**" because they assume that ***little interference*** will occur and hence that there is ***no need to do checking*** during transaction execution.
  - The **optimistic** protocol we describe uses transaction **timestamps** and also requires that the **write_sets** and **read_sets** of the transactions be **kept** by the system.
- In addition, ***start*** and ***end*** times for some of the **three phases** need to be kept for each transaction.
- Recall that the **write_set** of a transaction is the *set of items it writes*, and the **read_set** is the *set of items it reads*.
  - In the validation phase for transaction T_i, the protocol checks that T_i does not interfere with any committed transactions or with any other transactions currently in their validation phase.
- The **validation phase** for **T_i checks** that, for each such **transaction T_j** that is either **committed** or is in its *validation phase*, *one* of the following conditions ***holds***:
  1. Transaction T_j **completes** its ***write*** phase **before** T_i starts its ***read*** phase.
  2. T_i **starts** its **write** phase **after** T_j completes its **write** phase, and the ***read_set*** of T_i has no items in common with the ***write_set*** of T_j.
  3. Both the **read_set** and **write_set** of T_i have no items in common with the write_set of T_j, and T_j completes its read phase before T_i completes its read phase.
- When validating transaction T_i, the first condition is checked first for each transaction T_j, since (1) is the simplest condition to check.
- Only if condition (1) is false is condition (2) checked, and only if (2) is false is condition (3)—the most complex to evaluate—checked.
- If any one of these three conditions holds, there is no interference and Ti is validated successfully.
- If none of these three conditions holds, the validation of transaction Ti fails and it is aborted and restarted later because interference may have occurred.

3.5 Granularity of Data Items

- **All concurrency control techniques** assumed that the **database** was formed of a number of ***named data items***. A ***database item could*** be chosen to be one of the following:

- A database record.
- A field value of a database record.
- A disk block.
- A whole file.
- The whole database

♣ The **granularity** can affect the ***performance*** of **concurrency control** and **recovery**.

3.5.1 Granularity Level Considerations for Locking

♣ The size of **data items** is often called the **data item granularity**.

♣ **Fine** granularity **refers** to **small item sizes**, whereas **coarse** granularity **refers** to **large item sizes**.

- Several **tradeoffs** must be considered in choosing the data item size.
- We shall discuss data item size in the context of locking, although similar arguments can be made for other concurrency control techniques.

♣ **First**, notice that the larger the **data item size is**, the **lower the degree of concurrency permitted**.

- For example, if the data item size is a disk block, a transaction T that needs to lock a record B must lock the whole disk block X that contains B because a lock is associated with the whole data item (block).
- Now, if another transaction S wants to lock a different record C that happens to reside in the same block X in a conflicting lock mode, it is forced to wait. If the data item size was a single record, transaction S would be able to proceed, because it would be locking a different data item (record).

♣ **On the other hand**, the **smaller** the **data item** size is, the **more the number of items** in the **database**.

- Because every item is associated with a lock, the system will have a larger number of active locks to be handled by the lock manager.
- More lock and unlock operations will be performed, causing a higher overhead. In addition, more storage space will be required for the lock table. For timestamps, storage is required for the read_TS and write_TS for each data item, and there will be similar overhead for handling a large number of items.

♣ Given the above **tradeoffs**, an obvious question can be asked: ***What is the best item size***?

- The answer is that it depends on the types of transactions involved.
- If a typical transaction accesses a **small number of records**, it is **advantageous** to have the *data item granularity be one record*.

- On the other hand, if a **transaction** typically accesses **many records** in the **same** file, it may be better to have ***block or file granularity*** so that the transaction will consider all those records **as one (or a few) data items**.

3.6 Using Locks for Concurrency Control in Indexes

- **Two-phase locking** can also be **applied to indexes**, where the **nodes** of an **index correspond** to **disk pages**.
  - However, holding locks on index pages until the **shrinking** phase of 2PL (*Two-phase locking*) could cause an **undue** amount of transaction **blocking**.
  - This is because **searching** an **index** always ***starts at the root***, so if a transaction wants to **insert a record** (write operation), the **root** would be **locked** in **exclusive mode**, so all other ***conflicting lock requests*** for the index must wait until the **transaction enters** its **shrinking** phase.
- This ***blocks*** all other transactions from ***accessing the index***, so in practice other approaches to **locking** an **index** must be used.
- The **tree structure** of the **index** can be **taken advantage** of when developing a ***concurrency control scheme***.
  - For example, when an **index search** (**read** operation) is being executed, a ***path in the tree is traversed*** from the **root** to a **leaf**.
    - **Once** a **lower-level node** in the path has been **accessed**, the **higher-level nodes** in that path **will not be used** again.
    - So once a ***read lock on a child node*** is obtained, the lock on the ***parent can be released***.
  - **Second**, when an **insertion** is being applied to a **leaf** node (that is, when a **key** and a **pointer** are inserted), then a ***specific leaf node*** must be **locked** in **exclusive** mode.
  - However, if that node is **not full**, the insertion will ***not cause changes*** to ***higher-level index nodes***, which implies that they ***need not*** be *locked exclusively*.
- A ***conservative approach*** for insertions would be to lock the **root node** in exclusive mode and then to access the appropriate child node of the root.
- If the ***child node is not full***, then the lock on ***the root node*** can be **released**. This approach can be applied all the way down the **tree to the leaf**, which is typically **three** or ***four*** levels ***from the root***.
- Although ***exclusive locks are held***, they are soon **released**. An alternative, more optimistic approach would be to request and hold shared locks on the nodes leading to the leaf node, with an exclusive lock on the leaf.
  - If the **insertion** causes the leaf to split, insertion will propagate to a higher level node(s).
  - ***Then***, the locks on the higher level node(s) can be upgraded to exclusive mode.

- **Another** approach to ***index locking*** is to use a variant of the **B+-tree**, called the **B-link tree**.
  - In a ***B-link tree***, sibling nodes on the **same** level are **linked** together at **every level**.
  - This allows **shared locks** to be used when requesting a page and requires that the lock be released before ***accessing the child*** node.
  - For an insert operation, the **shared lock** on a node would be **upgraded** to **exclusive** mode. If a **split** occurs, the **parent** node must be **relocked** in exclusive mode.
  - One complication is for ***search operations*** executed concurrently with the **update**.
- Suppose that a ***concurrent update*** operation follows the *same path as the search*, and **inserts** a new entry ***into the leaf*** node.
  - In addition, suppose that the **insert** causes that **leaf** node to **split**. When the insert is **done**, the **search** process resumes, following the pointer to the desired leaf, only to find that the key it is looking for is not present because the split has moved that key into a new leaf node, which would be the right sibling of the original leaf node.
  - However, the search process can ***still succeed if it follows the pointer (link)*** in the *original leaf node* to its right sibling, where the ***desired key has been*** moved.
- Handling the **deletion** case, where ***two or more nodes*** from the ***index tree merge***, is also ***part of the B- link*** tree concurrency protocol. In this case, locks on the nodes to be merged are held as well as a lock on the parent of the two nodes to be merged.

CHAPTER – FOUR: Database Recovery Techniques

Database Recovery Techniques

4.1 Recovery Concepts

Recovery Outline and Categorization of Recovery Algorithms

- Recovery from transaction failures usually means that the database is ***restored*** to the *most* ***recent*** *consistent state* just before the time of failure.

- To do this, the system must *keep information* about the changes that were applied to data items by the various transactions.
- This **information** is typically kept in the **system log,** as we already discussed it on our last class sessions.

♣ A typical **strategy** for recovery may be summarized informally as follows:

1. If there is **extensive damage** to a wide portion of the database due to **catastrophic** *failure*, such as a **disk crash**, the recovery method restores a past copy of the database that was ***backed up*** to archival storage (typically tape) and reconstructs a more current state by **reapplying** or ***redoing*** the operations of committed transactions from the ***backed up*** **log**, up to the *time of failure*.

2. When the database is **not physically damaged** but has become *inconsistent due to* ***non-catastrophic*** failures (like *transaction or system error, Local errors or exception conditions*), the strategy is to ***reverse*** any changes that caused the inconsistency by ***undoing*** *some operations*.

It may also be necessary to ***redo*** some operations in order to restore a consistent state of the database, as we shall see. In this case we ***do not need*** a complete **archival** copy of the database. Rather, the **entries kept** in the *on-line system log* are consulted during recovery.

♣ Conceptually, we can distinguish **two main techniques** for recovery from **non-catastrophic** transaction failures:

(1) **Deferred** update and

(2) **Immediate** update.

♣ The **deferred update** techniques **do not** physically update the database on disk until ***after*** a transaction reaches *its commit point*; then the updates are recorded in the database.

- Before reaching commit, all transaction updates are recorded in the local transaction **workspace** (or **buffers**). *During commit*, the updates are first ***recorded persistently*** in the ***log*** and then ***written*** to the ***database***.
- If a transaction **fails** before reaching its commit point, it will *not have changed* the database in any way, so **UNDO** is *not needed*.
- It may be necessary to **REDO** the effect of the operations of a *committed transaction* from the **log**, because their effect may ***not yet*** have been ***recorded*** in the database. Hence, deferred update is ***also known*** as the ***NO-UNDO/REDO*** *algorithm***.** We will discuss this later.

♣ In the **immediate update** techniques, the database may be updated by ***some operations*** of a transaction ***before*** the transaction **reaches** its **commit** point. However, these operations are typically recorded in the **log** ***on disk*** by force **writing** ***before*** they are applied to the database, making recovery still possible.

- If a transaction **fails** *after recording* some changes in the database but *before reaching* its commit point, the **effect** of its operations on the database must

be **undone**; that is, the transaction *must be rolled back*.
- In the general case of immediate update, both ***undo*** and ***redo*** may be required during recovery. This technique, known as the **UNDO/REDO algorithm,** requires both operations, and is used most often in practice.
- A variation of the algorithm where all updates are recorded in the database before a transaction commits requires ***undo*** **only**, so it is known as the **UNDO/NO-REDO algorithm.** We will discuss these techniques later.

Caching of Disk Blocks

- The **recovery process** is often closely **intertwined** with *operating system functions*—in particular, the **buffering** and **caching** of disk pages in **main memory**.
- Typically, *one* or *more disk pages* that include the data items to be **updated** are **cached** into main memory **buffers** and then ***updated*** in memory *before being written back to disk*.
  - The caching of disk pages is **traditionally** an *operating system function*, but because of its importance to the **efficiency** of **recovery procedures**, it is handled by the DBMS by *calling low-level operating systems* routines.
  - In general, it is convenient to consider **recovery** in *terms* of the **database disk pages** (**blocks**). Typically a collection of **in-memory buffers**, called the **DBMS cache,** is kept under the *control of the DBMS* for the purpose of holding these buffers.
- A **directory** for the cache is used to keep track of which database items are in the **buffers**.
  - This can be a **table** of <disk page address, buffer location>entries.
  - When the DBMS requests action on some item, it ***first checks*** the *cache directory* to determine whether the disk page containing the item is in the cache.
  - **If it is not**, then the item must be *located on disk*, and the appropriate disk pages are *copied into the cache*.
- It may be necessary to **replace** (or **flush**) some of the cache buffers to **make space** available *for the new item*.
  - Some **page-replacement** strategy from operating systems, such as *least recently used* (**LRU**) or *first-in-first-out* (**FIFO**), can be used to select the buffers for replacement
- Associated with **each buffer** in the cache is a **dirty bit,** which can be *included in the directory entry*, to indicate ***whether*** or ***not the buffer*** has been **modified**.
  - When a page is **first read** from the *database* disk into a **cache buffer**, the cache **directory** is updated with the *new disk page address*, and the *dirty bit is set to 0* (zero).
  - As soon as the buffer is **modified**, the *dirty bit* for the corresponding directory entry is **set to 1** (one).
  - When the buffer contents are **replaced (flushed)** from the cache, the contents must first be written back to the corresponding disk page ***only if its dirty bit is 1***.
  - Another bit, called the **pin-unpin** bit, is also needed—a page in the cache is

pinned (bit **value 1** (one)) *if it cannot be written back* to disk as yet.

- **Two main strategies** can be employed when **flushing** a modified buffer back to disk.
  - The **first strategy**, known as **in-place updating**, writes the buffer back to the ***same original disk location***, thus overwriting the old value of any changed data items on disk. **Hence**, a ***single copy*** of each database disk block is maintained.
  - The **second strategy**, known as **shadowing**, writes an updated buffer at a different disk location, so multiple versions of data items can be maintained.
    - In general, the **old value** of the data item before updating is called the **before image (BFIM),** and the **new value** after updating is called the **after image (AFIM).**
    - In **shadowing**, both the **BFIM** and the **AFIM** can be kept on disk; hence, *it is not strictly necessary* to maintain a log for recovering.

4.2 Recovery Techniques Based on Deferred and Immediate Updates

4.2.1 Recovery Techniques Based on Deferred Update

- The **idea behind** deferred update techniques is to **defer or postpone** any actual updates to the database *until the transaction completes* its execution **successfully** and *reaches its commit* point.
- During transaction execution, the ***updates*** *are recorded* ***only*** *in the log* and in ***the cache*** buffers. After the transaction *reaches its commit* point and the ***log is force-written*** to disk, the updates are recorded in the database.
  - If a transaction fails before reaching its commit point, there is ***no need to undo any operations***, because the transaction has not affected the database on disk in any way.
  - Although this may **simplify recovery**, it *cannot be used in practice* unless transactions are *short and each transaction changes few items*. For other types of transactions, there is the potential for running ***out of buffer space*** because transaction changes must be held in the **cache buffers** until the commit point.

Recovery Using Deferred Update in a Single-User Environment

- In such **an environment**, the recovery algorithm can be rather simple. The algorithm **RDU_S** (Recovery using Deferred Update in a Single-user environment) uses a **REDO procedure**, given subsequently, for redoing certain write_item operations; it works as follows:
- **PROCEDURE RDU_S:** Use two lists of transactions:
  - The **committed transactions** since the last checkpoint, and
  - The **active transactions** (at most one transaction will fall in this category, because the system is single-user).

- Apply the **REDO** operation to all the **write_item** operations of the committed transactions from the log in the order in which they were written to the log. **Restart** the **active transactions**.

Deferred Update with Concurrent Execution in a Multiuser Environment

- For **multiuser systems** with concurrency control, the recovery process may be **more complex**, depending on the **protocols used for concurrency** control. In many cases, the concurrency control and recovery processes are interrelated.
- In general, the **greater the degree of concurrency** we wish to achieve, the more **time consuming the task of recovery** becomes.
- Consider a system in which concurrency control uses **strict two-phase** locking, so the locks on items remain in effect ***until the transaction reaches its commit point***. After that, the locks can be released.
  - This ensures **strict** and **serializable** schedules. Assuming that [**checkpoint**] entries are included in the log, a ***possible recovery algorithm*** for this case, which we call **RDU_M** (Recovery using Deferred Update in a Multi-user environment), is given next.
  - This procedure uses the REDO procedure defined earlier.
- **PROCEDURE RDU_M** (WITH CHECKPOINTS): Use two lists of transactions maintained by the system:
  - The **committed transactions** *T* since the last checkpoint **(commit list),** and
  - The **active transactions** T (**active list**).
- REDO all the WRITE operations of the committed transactions from the log, *in the order in which they were written into the log*. The transactions that are active and did not commit are effectively canceled and must be resubmitted.

4.2.2 Recovery Techniques Based on Immediate Updates

- In these techniques, when a transaction issues an update command, the database can be updated "**immediately**," without any need to wait for the transaction to reach its commit point. In these techniques, **however**, an **update** operation must still be ***recorded in the log*** (on disk) *before* it is applied to the database—using the write-ahead logging protocol—so that we can **recover in case of failure**.
- Provisions must be made for ***undoing*** the effect of update operations that have been applied to the database by a ***failed transaction***.
  - This is accomplished by ***rolling back*** the transaction and undoing the effect of the transaction's write_itemoperations.
- Theoretically, we can distinguish **two main categories** of immediate update algorithms.

- If the recovery technique ensures that all updates of a transaction are recorded in the database on disk ***before the transaction commits,*** there is **never a need** to REDO any operations of committed transactions.
  - This is called the **UNDO/NO-REDO recovery algorithm.**
- On the other hand, if the transaction is allowed to commit before all its changes are written to the database, we have the **most general** case, known as the **UNDO/REDO recovery algorithm.**

♣ This is also the most complex technique.

4.3 Shadow Paging and The ARIES Recovery Algorithm

4.3.1 Shadow Paging

♣ This recovery scheme **does not require** the use of a log in ***a single-user environment***. In a ***multiuser environment***, a log may be needed for the concurrency control method.

♣ **Shadow paging** considers the database to be **techniques**, made up of a ***number of fixed-size disk pages*** (or disk blocks)—say, ***n***—for recovery purposes.

- A **directory** with *n* entries is constructed, where the ***i^{th} entry points*** to the ***i^{th} database page*** on disk.
- The directory is **kept** in **main memory** if it ***is not too large***, and all references—reads or writes—to database pages on disk go through it.
- When a transaction begins executing, the **current directory**—whose entries point to the most recent or ***current database pages*** on disk—is copied into a **shadow directory.**
- The **shadow directory** is then *saved on disk* while the ***current directory*** is used by the **transaction**.

♣ During **transaction execution**, the ***shadow directory is never modified***. When a write_item operation is performed, a **new copy of the modified** database page is created, but the **old copy of that page is *not overwritten*.**

♣ **Instead**, the ***new page is written elsewhere***—on some previously unused disk block.

- The ***current directory*** entry is modified to point to the **new disk block**, whereas the **shadow directory** is **not** modified and continues to point **to the old unmodified** disk block.
- For pages updated by the transaction, **two versions are kept**. The **old version** is referenced by the **shadow directory**, and the **new version** by the **current directory.**

♣ To recover from a failure during transaction execution, it is sufficient to **free the modified database pages** and to **discard the current directory**.

♣ The state of the database before transaction execution is available through the

shadow directory, and that state is recovered by **reinstating the shadow directory**.

- The database thus is returned to its **state prior** to the transaction that was executing when the **crash occurred**, and **any modified pages are discarded**.
- **Committing a transaction** corresponds to ***discarding the previous shadow directory***. Since recovery involves **neither undoing nor redoing data items**, this technique can be categorized as a **NO-UNDO/NO-REDO** technique for recovery.
- In a **multiuser environment** with concurrent transactions, **logs and checkpoints must be incorporated** into the **shadow paging technique**.
  - **One disadvantage** of shadow paging is that the **updated database pages** change **location on disk**.
  - This makes it ***difficult to keep related database pages*** close together on disk without ***complex storage management*** strategies.
  - **Furthermore**, if the directory is **large**, the **overhead** of writing shadow directories to disk as transactions commit is significant.
- A further complication is how to handle **garbage collection** when a transaction commits. The old **pages referenced** by the shadow directory that have been updated must be released and added to a **list of free pages** for **future use**.
- *These pages are no longer needed* **after** the transaction commits. Another issue is that the operation to **migrate** between current and shadow directories must be implemented as **an atomic operation**.

4.3.2 The ARIES Recovery Algorithm

- We now describe the **ARIES algorithm** as an example of a recovery algorithm used in **database systems**. ARIES is based on **three concepts**:
  (1) Write- ahead logging,
  (2) Repeating history during redo, and
  (3) Logging changes during undo.
- The **ARIES recovery** procedure consists of **three main steps**:
  (1) Analysis,
  (2) REDO and
  (3) UNDO.
- The **analysis step** identifies the **dirty (updated) pages** in the buffer, and the ***set of transactions active*** at the time of the crash. The **appropriate** point in the **log** where

the **REDO** operation should start is also determined.

- ♣ The **REDO phase** actually ***reapplies updates*** from the **log** to the database.
  - Generally, the REDO operation is applied to ***only committed transactions***.
  - However, in **ARIES**, this is **not the case**. Certain information in the ARIES log will provide the ***start point*** for **REDO**, from which **REDO** operations are applied until the ***end of the log*** is reached. In addition, information stored by ARIES and in the data pages will allow ARIES to determine whether the operation to be redone has actually been applied to the database and hence need not be reapplied.
  - Thus *only the necessary REDO operations* are applied during recovery.
- ♣ Finally, during the **UNDO phase,** the log is **scanned backwards** and the operations of transactions that were active at the time of the crash are ***undone in reverse order***.
  - The information needed for ARIES to accomplish its recovery procedure includes the log, the ***Transaction Table***, and the ***Dirty Page Table***. In addition, **checkpointing** is used. These **two tables** are **maintained** by the **transaction manager** and written to the log during checkpointing.
- ♣ In ARIES, every log record has an associated **log sequence number (LSN)** that is *monotonically increasing* and indicates the address of the log record on disk.
  - **Each** LSN corresponds to a ***specific change*** (action) of some transaction.
  - In addition, **each** data page will store the LSN of the *latest log record corresponding to a change for that page*.
- ♣ A log record is written for any of the following actions:
  1. Updating a page (write),
  2. Committing a transaction (commit),
  3. Aborting a transaction (abort),
  4. Undoing an update (undo), and
  5. Ending a transaction (end).
- ♣ The need for including the **first three** actions in the log has been discussed, but the last **two** need some explanation.
- ♣ When an update is **undone**, a ***compensation log record*** is written in the log. When a transaction **ends**, whether by committing or aborting, an ***end log record*** is written.
- ♣ *Common fields in all log records* include:
  (1) The previous LSN for that transaction,
  (2) The transaction ID, and
  (3) The type of log record.
  The previous LSN is important because it links the log records (in reverse order) for each transaction. For an **update (write**) action, *additional fields in the log record* include:
  (4) The page ID for the page that includes the item,
  (5) The length of the updated item,
  (6) Its offset from the beginning of the page,
  (7) The before image of the item, and
  (8) Its after image.
- ♣ Besides the log, **two tables** are needed for **efficient** recovery:
  1) The **Transaction Table** and

2) The **Dirty Page Table**

- Which are maintained by the ***transaction manager***.
- When a crash occurs, these tables are **rebuilt** in the *analysis phase* of recovery.
- The **Transaction Table** contains an ***entry for each active transaction***, with information such as the *transaction ID*, *transaction status*, and the *LSN* of the most recent log record for the transaction.
- The **Dirty Page Table** contains an ***entry for each dirty page*** in the buffer, which includes the **page ID** and the **LSN** corresponding to the earliest update to that page.

♣ **Checkpointing** in ARIES consists of the following:
(1) Writing a begin_checkpoint record to the **log**,
(2) Writing an end_checkpoint record to the **log**, and
(3) Writing *the LSN of* the begin_checkpoint record to a **special file**.

- This **special file** is *accessed during recovery* to locate the *last checkpoint* information.
- With the **end_checkpoint** record, the contents of both the *Transaction Table* and *Dirty Page Table* are appended to the end of the log.
- To reduce the cost, **fuzzy checkpointing** is used so that the DBMS can continue to execute transactions during checkpointing.
- In addition, the contents of the DBMS cache do not have to be flushed to disk during checkpoint, since the Transaction Table and Dirty Page Table—which are appended to the log on disk—contain the information needed for recovery.
- Notice that if a crash occurs during **checkpointing**, the special file will refer to the previous checkpoint, which is used for recovery.

♣ After a crash, the ***ARIES recovery manager*** takes over. Information from the last checkpoint is first accessed through the **special file**. The **analysis phase** starts at the begin_checkpoint record and proceeds to the end of the log. When the end_checkpoint record is encountered, the Transaction Table and Dirty Page Table are accessed (recall that these tables were written in the log during checkpointing). During analysis, the log records being analyzed may cause modifications to these two tables.

4.4 Recovery in Multidatabase Systems

♣ So far, we have implicitly assumed that a transaction accesses a **single database**. In some cases a **single transaction**, called a **multidatabase transaction**, may require access to **multiple databases**.

- These databases may **even** be stored on ***different types of DBMSs***; for example, some DBMSs may be **relational**, whereas others are **object-oriented**, **hierarchical**, or **network** DBMSs.
- In such a case, each DBMS involved in the ***multidatabase*** transaction may have ***its own recovery technique*** and transaction manager separate from those of the other DBMSs.

♣ This situation is somewhat **similar** to the case of a **distributed database** management system, where parts of the database reside at **different sites** that are connected by a communication network.

- To maintain the **atomicity** of a **multidatabase** transaction, it is necessary to have a ***two-level recovery mechanism***.
- A **global recovery manager**, or **coordinator**, is needed to maintain information needed for recovery, in addition to the ***local recovery managers*** and the ***information*** they maintain (**log**, **tables**).
- The ***coordinator usually follows*** a protocol called the **two-phase commit protocol**, whose **two phases** can be stated as follows:

  Phase 1: When all ***participating databases*** signal the coordinator that the part of the multidatabase transaction involving each has concluded, the coordinator sends a message "***prepare for commit***" to each participant to get ready for committing the transaction.
  - ✓ Each participating database receiving that message will **force-write all log** records and needed information for local recovery to disk and then send a "**ready to commit**" or "**OK**" signal to the **coordinator**.
  - ✓ If the **force-writing** to disk fails or the local transaction cannot commit for some reason, the participating database sends a "cannot commit" or "not OK" signal to the coordinator. If the coordinator does not receive a reply from a database within a certain time out interval, it assumes a "**not OK**" response.

  Phase 2: If ***all*** **participating** databases reply "**OK**," and the coordinator's vote is also "**OK**," the transaction is **successful**, and the coordinator sends a "**commit**" signal for the transaction to the **participating databases**.
  - ✓ *Because all the local effects* of the transaction and information needed for local recovery have been recorded in the logs of the participating databases, recovery from **failure is now possible**.
  - ✓ Each participating database completes transaction commit by writing a [**commit**] entry for the transaction in the log and permanently updating the database if needed.
  - ✓ On the other hand, if one or more of the participating databases or the coordinator have a "**not OK**" response, the transaction has failed, and the coordinator sends a message to "roll back" or UNDO the local effect of the transaction to each participating database.
  - ✓ This is done by undoing the transaction operations, using the log.
- The **net effect of the two-phase commit protocol** is that either **all participating databases commit** the effect of the transaction or **none** of them do.
- In case any of the participants—or the **coordinator**—fails, it is always possible to recover to a state where either the transaction is committed or it is **rolled back**.
- A **failure** during or before **Phase 1** usually requires the transaction to be ***rolled back***, whereas a **failure** during **Phase 2** means that a **successful** transaction can **recover** and **commit**.

CHAPTER – FIVE: DISTRIBUTED DATABASE SYSTEM

Distributed Database System

5.1 Distributed Database Concepts, Data Fragmentation, Replication, and Allocation Techniques for Distributed Database Design

- In this chapter we turn our attention to **distributed databases** (**DDBs**), distributed database management systems (**DDBMSs**), and *how the client-server architecture* is used as a platform for database application development.
- The **DDB technology** emerged as a **merger** of **two** technologies:
  (1) Database technology, and
  (2) Network and data communication technology.
  - ✓ The latter has made tremendous strides in terms of **wired** and **wireless technologies**—from satellite and cellular communications and Metropolitan Area Networks (MANs) to the standardization of protocols like Ethernet, TCP/IP, and the Asynchronous Transfer Mode (ATM) as well as the explosion of the Internet, including the newly started Internet-2 development.
  - ✓ While early databases moved toward ***centralization and resulted in monolithic gigantic databases*** in the **seventies** and early **eighties**, the trend reversed toward more decentralization and autonomy of processing in the late eighties.
- With advances in **distributed processing and distributed computing** that occurred in the operating systems arena, the database research community did considerable work to address the issues of data distribution, distributed query and transaction processing, distributed database metadata management, and other topics, and developed many research prototypes.
- However, a full-scale comprehensive DDBMS that implements the functionality and techniques proposed in DDB research never emerged as a commercially viable product. Most major vendors redirected their efforts from developing a "pure" DDBMS product into developing systems based on client-server, or toward developing active heterogeneous DBMSs.
- Organizations, however, have been very interested in the ***decentralization*** of processing (at the system level) while achieving an ***integration*** of the information resources (at the logical level) within their geographically distributed systems of databases, applications, and users. Coupled with the advances in communications, there is now a general endorsement of the client-server approach to application development, which assumes many of the DDB issues.
- Distributed databases bring the advantages of distributed computing to the database management domain. A **distributed computing system** consists of a number of processing elements, ***not necessarily homogeneous***, that are interconnected by a computer network, and that cooperate in performing certain assigned tasks. As a general goal, *distributed computing systems* partition a big, unmanageable problem into smaller pieces and solve it efficiently in a coordinated manner.

- The ***economic viability*** of this approach **stems from two** reasons:
  (1) More ***computer power*** is harnessed to solve a complex task, and
  (2) Each **autonomous** processing element can be managed independently and develop its own applications.
- We can define a **distributed database (DDB)** as a collection of multiple logically interrelated databases distributed over a computer network, and a ***distributed database management system (DDBMS)*** as a software system that manages a distributed database while making the distribution transparent to the user. A collection of files stored at different nodes of a network and the maintaining of inter relationships among them via hyperlinks has become a common organization on the Internet, with files of Web pages.
- The common functions of database management, including uniform query processing and transaction processing, ***do not*** apply to this scenario yet. The technology is, however, moving in a direction such that distributed ***World Wide Web*** (**WWW**) databases will become a reality in the near future.

5.1.1 Advantages of DDBs, Parallel Vs Distributed Technology

- There are **two main types** of **multiprocessor** system architectures that are **commonplace**:
  1) **Shared memory (tightly coupled) architecture***:* Multiple processors **share** secondary (**disk**) storage and also share **primary memory.**
  2) **Shared disk (loosely coupled) architecture***:* Multiple processors ***share*** secondary (***disk***) storage ***but*** each has ***their own primary*** memory.
- These architectures enable processors to ***communicate*** without the ***overhead*** of ***exchanging messages*** over a network.
- Database management systems developed using the above types of architectures are termed **parallel database management systems** rather than **DDBMS**, since they utilize ***parallel processor technology***.
- Another type of multiprocessor architecture is called **shared nothing architecture.**
  - In this architecture, ***every processor has its own primary and secondary*** (disk) memory, ***no common memory exists***, and the processors communicate over a ***high-speed interconnection*** network (bus or switch).
  - Although the **shared nothing** architecture resembles a distributed database computing environment, ***major differences*** exist in the *mode of operation*.
- In shared nothing multiprocessor systems, there is **symmetry** and **homogeneity** of **nodes**; this is ***not true of the distributed*** database environment where **heterogeneity** of hardware and operating system at each node is very common.
- Distributed database management has been proposed for **various reasons** ranging from organizational **decentralization** and **economical processing** to greater **autonomy**.

♣ We highlight some of these advantages here.

1) Management of distributed data with different levels of transparency: Ideally, a DBMS should be **distribution transparent** in the sense of hiding the details of where each file (table, relation) is physically stored within the system. The following types of transparencies are possible:

- *Distribution or network transparency:* This refers to freedom for the user from the operational details of the network. *It may be divided into location transparency and naming transparency.*
  - **Location transparency** refers to the fact that the command used to perform a task is **independent** of the **location** of data and the location of the **system** where the command was issued.
  - **Naming transparency** implies that once a **name** is specified, the named **objects** can be **accessed unambiguously** without additional specification.
- *Replication transparency:* copies of data may be stored at multiple sites for better availability, performance, and reliability. Replication transparency makes the user **unaware** of the **existence** of **copies**.
- *Fragmentation transparency:* **Two** types of fragmentation are possible.
  - **Horizontal fragmentation** distributes a relation into sets of tuples (rows).
  - **Vertical fragmentation** distributes a relation into subrelations where each subrelation is defined by a subset of the columns of the original relation.
  - A global query by the user must be transformed into several fragment queries.

2) Increased reliability and availability: These are **two** of the **most common** potential advantages cited for **distributed databases**. **Reliability** is broadly defined as the probability that a system is running (not down) at a certain time point, whereas **availability** is the probability that the system is continuously available during a time interval.

3) Improved performance: A distributed DBMS fragments the database by keeping the data closer to where it is needed most.

4) Easier expansion: In a distributed environment, expansion of the system in terms of adding more data, increasing database sizes, or adding more processors is much easier.

5.1.2 Data Fragmentation, Replication and Allocation Techniques

Data Fragmentation:

- Before we decide on how to distribute the data, we must determine the *logical units*, *called* **fragments** of the database that are to be distributed.
  - The simplest logical units are the ***relations*** themselves; that is, each ***whole*** relation is to be stored at a *particular site*. Example, we must decide on a site to store each of the relations EMPLOYEE, DEPARTMENT, PROJECT, WORKS_ON, and DEPENDENT of last semester.
  - In many cases, however, a **relation** can be divided into smaller logical units for distribution. We may want to store the database information relating to each department at the computer site for that department. A technique called ***horizontal fragmentation*** can be used to partition each relation by department.
- A **horizontal fragment** of a relation is a **subset** of the **tuples** in that relation. The tuples that belong to the *horizontal fragment* are specified by a condition on one or more attributes of the relation. Often, **only** a single attribute is involved.
  - For example, we may define *three horizontal fragments* on the EMPLOYEE relation with the following conditions: (DNO = 5), (DNO = 4), and (DNO =1)—each fragment contains the EMPLOYEE tuples working for a particular department.
  - Similarly, we may define *three horizontal fragments* for the PROJECT relation, with the conditions (DNUM = 5), (DNUM= 4), and (DNUM = 1)—each fragment contains the PROJECT tuples controlled by a particular department.
- **Horizontal fragmentation** divides a relation "**horizontally**" by grouping rows to create *subsets of tuples*, where each subset has a certain logical meaning.
  - These fragments can then be assigned to different sites in the distributed system.
- **Derived horizontal fragmentation** applies the **partitioning** of a **primary** relation (DEPARTMENT in our example) to other **secondary** relations (EMPLOYEE and PROJECT in our example), which are related to the primary **via a foreign** key.
  - This **way**, related data between the **primary** and the **secondary** relations gets fragmented in the **same way**.
- **Vertical fragmentation** divides a relation "**vertically**" by **columns**.
- A **vertical fragment** of a relation **keeps only certain attributes** of the relation.
  - For example, we may want to fragment the EMPLOYEE relation into two vertical fragments.
  - The **first** fragment includes *personal information*—NAME, BDATE, ADDRESS, and SEX—and the **second** includes *work-related information*— SSN, SALARY, SUPERSSN, DNO.
  - This vertical fragmentation is ***not quite proper*** because, if the two fragments are stored separately, we cannot put the original employee tuples back together, since there is *no common attribute* between the two fragments. It is necessary to include the primary key or some candidate key attribute in ***every***

vertical fragment so that the full relation can be reconstructed from the fragments. Hence, we must add the SSN attribute to the personal information fragment.

- ♣ We can intermix the **two types of fragmentation**, yielding a **mixed (Hybrid) fragmentation.**
  - For example, we may ***combine*** the ***horizontal*** and ***vertical*** fragmentations of the EMPLOYEE relation given earlier ***into*** a ***mixed*** fragmentation that includes ***six fragments***.
  - In this case the *original* relation can be *reconstructed* by applying *UNION and OUTER UNION* (or OUTER JOIN) operations in the appropriate order.

Data Replication and Allocation:

- ♣ Replication is useful in improving the availability of data. The **most extreme** case is replication of the ***whole database*** at **every** site in the distributed system, thus creating a **fully replicated distributed database.**
  - This can **improve availability** remarkably because the system can continue to operate as long as at least one site is up.
  - It **also improves performance** of **retrieval** for **global** queries, because the result of such a query can be obtained locally from any one site; hence, a retrieval query can be processed at the local site where it is submitted, if that site includes a server module.
- ♣ The ***disadvantage*** of ***full*** replication is that it ***can slow down*** update operations ***drastically***, since a ***single logical*** update must be performed on ***every copy*** of the database to keep the copies ***consistent***.
- ♣ This is *especially true if many copies* of the database exist. **Full** replication **makes** the **concurrency** control and **recovery** techniques **more expensive than** they would be if there were no replication as we shall discuss later in 5.2.
- ♣ The other extreme from full replication involves having **no replication**—that is, each fragment is stored at ***exactly one site***.
  - In this case all fragments ***must be*** disjoint, **except** for the **repetition** of **primary keys** among vertical (or **mixed**) fragments. This is also called **no redundant allocation.**
- ♣ Between these **two extremes**, we have a wide spectrum of **partial replication** of the data—that is, some fragments of the database **may be** replicated whereas others **may not**. The number of copies of each fragment can range from one up to the total number of sites in the distributed system.
- ♣ A **special case** of **partial** replication is **occurring** heavily in applications where **mobile workers**—such as sales forces, financial planners, and claims adjustors—carry partially replicated databases with them on laptops and personal digital assistants and synchronize them periodically with the server database.
- ♣ A **description** of the replication of fragments is sometimes called a **replication schema.**

- **Each** fragment—or each **copy** of a **fragment**—must be assigned to a particular site in the distributed system.
  - This process is called **data distribution** (or **data allocation**).
- The **choice** of **sites** and the **degree** of replication depend on the **performance** and **availability** goals of the system and on the **types** and **frequencies** of transactions *submitted* at each site.
  - For example, if high availability is required and transactions can be submitted at any site and if most transactions are retrieval only, a fully replicated database is a good choice.
  - However, if ***certain transactions*** that access particular parts of the database are mostly submitted at a ***particular site***, the corresponding set of fragments can be allocated at ***that site only***. Data that is accessed at multiple sites can be replicated at those sites. If many updates are performed, it may be useful to limit replication.
- Finding an **optimal** or even a good solution to **distributed** data **allocation** is a **complex** optimization problem

5.2 Concurrency Control and Recovery in Distributed Databases

- For **concurrency control** and **recovery purposes**, numerous problems arise in a distributed DBMS environment that are **not encountered** in a **centralized DBMS** environment. These include the **following**:
  - *Dealing with* **multiple copies** *of the data items:*
    - The concurrency control method is **responsible** for **maintaining consistency** among these copies.
    - The recovery method is **responsible** for making a **copy** consistent with other copies if the site on which the copy is stored **fails** and **recovers** later.
  - *Failure of individual sites:*
    - The DDBMS **should** continue to operate with its **running sites**, if possible, when **one** or **more** individual sites **fail**.
    - When a site **recovers**, its **local** database **must** be brought up to date with the rest of the sites **before** it **rejoins** the system.
  - *Failure of communication links:*
    - The system must be able to deal with **failure** of **one** or **more** of the **communication links** that connect the sites.
    - An extreme case of this problem is that **network partitioning** may occur.
    - This **breaks** up the **sites** into **two** or more partitions, where the sites **within** each **partition** can communicate only with one another and not with sites in **other partitions**.
  - *Distributed commit:*
    - Problems can arise with committing a transaction that is accessing databases stored on multiple sites **if some sites fail during** the **commit** process.

        - The **two-phase commit protocol** (previous Chapters) is often used to deal with this problem.
    - *Distributed deadlock:*
        - **Deadlock** may occur among several sites, so techniques for dealing with deadlocks must be **extended** to take this into account.
- **Distributed concurrency** control and **recovery** techniques **must deal** with **these** and **other** problems.

5.3 Types of Distributed Databases

- The term distributed database management system can describe **various systems** that **differ** from **one another** in many **respects**.
- The main thing that all such systems have in common is the **fact that data** and **software** are **distributed** over **multiple** sites connected by some form of communication network.
- In this section we discuss a number of types of **DDBMSs** and the **criteria** and **factors** that make some of these systems different.
- The first factor we consider is the **degree of homogeneity** of the DDBMS software.
- If **all servers** (or individual local DBMSs) use **identical** software and **all users** (clients) use **identical** software, the DDBMS is called **homogeneous;** otherwise, it is called **heterogeneous.**
- Another factor related to the degree of homogeneity is the **degree of local autonomy.**
    - If there is no provision for the local site to function as a stand-alone DBMS, then the system has **no local autonomy.**
    - On the other hand, if ***direct access*** by local transactions to a server is permitted, the system has **some** degree of **local autonomy**.
- At one **extreme** of the **autonomy spectrum**, we have a DDBMS that "**looks like**" a centralized DBMS to the user.
    - A single conceptual schema exists, and all access to the system is obtained through a site that is part of the DDBMS—which means that **no local autonomy** exists.
- At the other **extreme** we encounter a type of DDBMS called a ***federated DDBMS*** (or a ***multidatabase*** *system*).
    - In such a system, ***each server*** is an ***independent*** and ***autonomous*** centralized DBMS that has its own ***local users***, ***local transactions***, and ***DBA*** and hence has a very ***high degree of local autonomy***.
- The term **federated database system (FDBS)** is used when there is some global view or **schema** of the **federation** of databases that is **shared** by the applications.
- On the other hand, a **multi-database system** does not have a ***global schema*** and ***interactively*** constructs ***one*** as ***needed*** by the application.
- **Both** systems are **hybrids** between **distributed** and **centralized** systems and the distinction we made between them is **not strictly** followed. We will refer to them as **FDBSs** in a generic sense.

- In a **heterogeneous FDBS**, **one server** may be a **relational** DBMS, another a **network** DBMS, and a third an **object** or **hierarchical** DBMS; in such a case it is necessary to have a **canonical** system language and to include **language translators** to translate subqueries from the canonical language to the language of each server.
- We briefly discuss the issues affecting the design of FDBSs below.

Federated Database Management Systems Issues:

- The type of **heterogeneity** present in **FDBSs** may arise from several sources. We discuss these sources first and then point out how the different types of autonomies contribute to a **semantic heterogeneity** that must be resolved in a heterogeneous FDBS.
  - ***Differences in data models:***
    - Databases in an organization come from a **variety** of data **models** including the **so-called legacy models** (network and hierarchical), the **relational** data model, the **object** data model, and **even** files.
    - The modeling capabilities of the **models vary**. Hence, to **deal** with them **uniformly** via a single global schema or to process them in a single language is challenging.
    - Even if **two** databases are **both** from the **RDBMS** environment, the **same information** may be represented as an attribute name, as a relation name, or as a value in different databases.
    - This **calls** for an **intelligent** query- processing mechanism that can relate information based on **metadata**.
  - ***Differences in constraints:***
    - **Constraint** facilities for specification and implementation **vary** from **system** to **system**. There are **comparable** features that must be reconciled in the construction of a **global schema**.
    - For example, the relationships from ER models are represented as referential integrity constraints in the relational model. Triggers may have to be used to implement certain constraints in the relational model. The global schema must also deal with potential conflicts among constraints.
  - ***Differences in query languages:***
    - Even with the same data model, the **languages** and their **versions** vary.
    - For example, SQL has multiple versions like SQL-89, SQL-92 (SQL2), and SQL3, and each system has its own set of data types, comparison operators, string manipulation features, and so on.

Semantic Heterogeneity:

- **Semantic heterogeneity** occurs when there are **differences** in the ***meaning***, ***interpretation***, and ***intended*** use of the **same** or **related** data.
- **Semantic heterogeneity** among component database systems (DBSs) creates the biggest hurdle in designing **global schemas** of heterogeneous databases. The **design autonomy** of component DBSs refers to their freedom of choosing the following design parameters, which in turn affect the eventual complexity of the FDBS:
  - ***The universe of discourse from which the data is drawn:*** For example, two customer accounts databases in the federation may be from United States and Japan with entirely different sets of attributes about customer accounts required by the accounting practices. Currency rate fluctuations would also present a problem. Hence, relations in these two databases which have identical names—CUSTOMER or ACCOUNT—may have some common and some entirely distinct information.
  - ***Representation and naming:*** The representation and naming of data elements and the structure of the data model may be prespecified for each local database.
  - ***The understanding, meaning, and subjective interpretation of data.*** This is a *chief* contributor to semantic heterogeneity.
  - ***Transaction and policy constraints:*** These deal with serializability criteria, compensating transactions, and other transaction policies.
  - ***Derivation of summaries:*** Aggregation, summarization, and other data-processing features and operations supported by the system.

5.4 Distributed DB in Oracle

- In the ***client-server architecture***, the Oracle database system is divided into **two** parts:
  (1) A **front-end** as the **client** portion, and
  (2) A **back-end** as the **server** portion.
- The client portion is the front-end database application that interacts with the user.
  - The client has no data access responsibility and merely handles the requesting, processing, and presentation of data managed by the server.
- The server portion runs Oracle and handles the functions related to concurrent shared access.
  - It accepts SQL and PL/SQL statements originating from client applications, processes them, and sends the results back to the client.
- Oracle client-server applications provide location transparency by making location of data transparent to users; several features like views, synonyms, and procedures contribute to this.
  - Global naming is achieved by using <tablename.@,databasename> to refer to tables uniquely.

- Oracle uses a **two-phase commit protocol** to deal with concurrent distributed transactions. The COMMIT statement **triggers** the **two-phase** commit mechanism.
- The **RECO** (recoverer) background process **automatically resolves** the outcome of those distributed transactions in which the commit was **interrupted**. The RECO of **each local** Oracle **Server** automatically **commits** or rolls **back** any "**in-doubt**" distributed transactions consistently on all involved nodes.
- For **long-term failures**, Oracle allows each **local DBA** to **manually commit** or **roll** back any **in-doubt** transactions and **free up** resources. Global consistency can be maintained by restoring the database at each site to a predetermined fixed point in the past.
- All **Oracle databases** in a distributed database system (DDBS) use Oracle's **networking** software **Net8** for **inter-database** communication.
  - **Net8** allows databases to **communicate** across networks to support **remote** and **distributed** transactions.
  - It **packages** SQL statements into **one** of the **many** communication **protocols** to facilitate client to server communication and **then packages** the results back similarly to the client.
  - Each database has a unique global name provided by a hierarchical arrangement of network domain names that is prefixed to the database name to make it unique.
- Oracle supports **database links** that define a **one-way** communication path from **one** Oracle **database** to **another**.
- For example:
  - **CREATE DATABASE LINK** sales.us.Tigray;
  - Establishes a **connection** to the **sales** database under the network domain us that comes **under** domain **Tigray**.
- Data in an Oracle DDBS can be replicated using snapshots or replicated master tables. Replication is provided at the following levels:
  - *Basic replication:* Replicas of tables are managed for read-only access. For updates, data must be accessed at a single primary site.
  - *Advanced (symmetric) replication:* This extends beyond basic replication by allowing applications to update table replicas throughout a replicated DDBS. Data can be read and updated at any site. This requires additional software called Oracle's advanced replication option.
- A **snapshot** generates a copy of a part of the table by means of a query called the ***snapshot defining query***. A simple snapshot definition looks like this:
  - **CREATE SNAPSHOT** sales.orders **AS SELECT** * **FROM** sales.us. Tigray;

CHAPTER – SIX: CONCEPTS FOR OBJECT ORIENTED DATABASES

Concepts for Object Oriented Databases

6.1) Overview of Object-Oriented Concepts

- **Traditional** data *models* and systems, such as ***relational***, ***network***, and ***hierarchical***, have been quite ***successful*** in developing the database technology required for many ***traditional business*** database applications.
- **However**, they have certain **drawbacks** when more **complex** database applications must be designed and implemented—for example,
  - Databases for engineering design and manufacturing (CAD/CAM and CIM
  - Scientific experiments,
  - Telecommunications,
  - Geographic information systems, and
  - Multimedia

-
- chapter